

Efficient Noncommutative KEMs from Twisted Dihedral Group Ring

Ali Raya^{a,*}, Vikas Kumar^b, Sugata Gangopadhyay^a, Aditi Kar Gangopadhyay^b

^aDepartment of Computer Science and Engineering, IIT Roorkee, Haridwar, 27667, Uttarakhand, India

^bDepartment of Mathematics, IIT Roorkee, Haridwar, 27667, Uttarakhand, India

Abstract

NTRU schemes have been extensively studied as post-quantum proposals within the category of lattice-based constructions. Numerous designs have been introduced with security assumptions based on the NTRU hard problem; some focused on security, and others were motivated by faster computations. Recently, some proposals for noncommutative NTRU have appeared in the literature, claiming more resistance to some algebraic attacks. While these proposals provide practical cryptosystems, they fail to perform similarly to the original NTRU over the ring of integers. This work introduces the first construction of noncommutative NTRU that matches the speed of NTRU over the ring of integers. Additionally, we present another construction over the ring of Eisenstein integers, demonstrating that performance can be further enhanced. We comprehensively implement the Key Encapsulation Mechanisms (KEMs) based on our constructions and compare their efficiency and compactness to both commutative and noncommutative NTRU variants in the literature. Our findings indicate that the new designs provide competitive memory and time requirements while utilizing noncommutative algebra. For example, our noncommutative KEM based on the twisted dihedral group ring over the ring of integers achieves encapsulation and decapsulation speeds comparable to NTRU-HPS, with a key generation speed that is 2.5 times faster. Additionally, our construction based on the ring of Eisenstein integers is at least 1.6 times faster for key generation and 1.3 times faster for both encapsulation and decapsulation compared to NTRU-HPS.

Keywords: NTRU, Twisted group rings, Noncommutative, Dihedral group

1. Introduction

Recent advances in research aimed at developing quantum computers have surged, with the ultimate goal of creating a large-scale quantum computer. The existence of such a powerful tool is expected to enable the efficient solving of some computational problems like never before. However, these advancements do not come without consequences; they pose a direct threat to the public key schemes currently in use. The mathematical foundations of existing cryptosystems rely on the difficulty of problems like Integer Factorization and the (Elliptic Curve) Discrete Logarithm Problem. These problems cannot be solved *efficiently* even with today's most advanced classical computers. This is not the case for a large-scale quantum computer that can solve these problems in polynomial time due to Shor's algorithm [1] and its modifications [2, 3]. As a result, various standardization bodies have started the process of selecting quantum-safe cryptosystems that rely on mathematical problems that cannot be solved efficiently, whether using a classical computer or a quantum one.

The National Institute of Standards and Technology (NIST) is leading the efforts to establish quantum-safe standards for cryptographic algorithms and has initiated this process in 2016 [4]. After three rounds of evaluation, NIST has selected several algorithms for standardization [5], including those for public-key encryption/key encapsulation mechanisms and signature schemes. The evaluation of other schemes, particularly signature schemes [6], is currently ongoing. Upon analyzing the submitted and selected algorithms, it is evident that the majority of submissions are focused on lattice-based cryptography. Lattice-based schemes demonstrate competitive time and memory efficiency compared to other cryptographic families. Additionally, their relatively straightforward design makes them particularly appealing for selection.

NTRU [7] is one of the earliest lattice-based cryptographic schemes, first introduced by Hoffstein, Pipher, and Silverman at Crypto'96. Its simple design has attracted significant attention from cryptanalysts. After nearly three decades of analysis, the security of the NTRU cryptosystem remains largely consistent with the original assessments, given well-chosen parameter sets. As a result, many variants of NTRU have emerged in the literature, with the aim of improving computational efficiency or enhancing resistance to different attacks. The confidence in the robustness of the NTRU assumption has led to several proposals advancing through the various rounds

*Corresponding author

Email addresses: ali_r@cs.iitr.ac.in (Ali Raya),
v_kumar@ma.iitr.ac.in (Vikas Kumar),
sugata.gangopadhyay@cs.iitr.ac.in (Sugata Gangopadhyay),
aditi.gangopadhyay@ma.iitr.ac.in (Aditi Kar Gangopadhyay)

of the NIST standardization process. Falcon [8], chosen for standardization by NIST as a signature scheme, relies on the difficult assumption of NTRU. Additionally, two NTRU-like KEMs reached the third round of NIST evaluation. These include NTRU KEM, which is a finalist [9] and combines NTRUEncrypt [10] and NTRU HRSS [11] from earlier rounds, and NTRUPrime [12], which is an alternate candidate that merges SNTRU Prime and NTRU LPrime. Further, NTRU has been selected for standardization by the IEEE as a quantum-safe scheme based on hard mathematical problems (IEEE-1363.1) [13].

Most of the introduced NTRU variants are based on commutative rings, with some minor modifications aimed at either reducing the attack surface, as seen in NTRU Prime [12], or enhancing performance with certain rings that facilitate the Number Theoretic Transform (NTT) multiplication, such as Falcon [8]. Generally, the computational hard assumption behind NTRU can be expressed as:

*Given the public key h from a ring $\mathcal{R}$ calculated as $h = f^{-1} * g \pmod{q}$, where the elements f, g are sampled from the underlying ring with small coefficients and q is a modulus, find f', g' with small coefficients such that $f' * h = g' \pmod{q}$.*

Other relaxed assumptions related to the NTRU assumption have been analyzed and used to construct various protocols. The NTRU learning problem [14, 4.4.4] serves as an example of a relaxed version of the NTRU assumption, which can be formulated as follows:

*Given many samples of h_i calculated as $h_i = f^{-1} * g_i$, where the same polynomial f is used with different g_i across the samples, find f .*

The NTRU learning assumption has been utilized to develop several protocols, as seen in [15] and [16]. This assumption was believed to be difficult; for instance, [16] states "It is not currently known whether there exists significantly better attacks against the NTRU learning problem than against the usual NTRU problem"

A study by Kim and Lee [17] demonstrated that the NTRU learning problem is not as difficult as previously thought. They found that the secret key f could be retrieved by obtaining n different samples of h_i , where n is the modulus degree used in the ring $\mathcal{R} = \mathbb{Z}[x]/(x^n - 1)$ from which f and g are sampled. The attack proposed by Kim and Lee is effective when the Hamming weight of the secret key is known and fixed. It relies on solving a linear system of equations that can be constructed due to the **commutativity** of the underlying ring. Later, Bai et al. [18] introduced an extension of this attack where no prior knowledge of the Hamming weight is necessary. Yet, it still relies on the commutativity of the ring to formulate the system of equations. Due to this, several noncommutative NTRU variants have been proposed, as they may provide greater resistance against such or similar algebraic attacks.

1.1. Previous and Related Works

A series of noncommutative variants of NTRU have appeared in the literature. Most of them have been proposed theoretically without concrete proof of their practicality, while others have been designed with assumptions different from NTRU's hard assumption.

Hoffstein and Silverman proposed the first noncommutative analog of NTRU [19] based on the group ring of the dihedral group. Their scheme introduced a different design than the original NTRU, which was later discovered to be susceptible to an attack [20] that breaks the new proposal of the noncommutative NTRU but not the original NTRU assumption. Truman analyzed the attack against the first version of noncommutative NTRU in [21]. The author showed the possibility of extending the attack to other related group rings that are close by definition to the one used by Hoffstein and Silverman.

NNRU [22] and PairTRU [23] introduced similar concepts of noncommutative NTRU with designs different from the original NTRU. Other works maintained the core hard assumption of NTRU but constructed the schemes over different noncommutative rings. QTRU [24] introduced NTRU over quaternion algebra, aiming to provide a more resilient variant against lattice attacks. The author claims that QTRU is more robust against such attacks, albeit with a performance cost that is four times slower than NTRU at equivalent security levels.

Inspired by QTRU, both SQTRU [25] and OTRU [26] have been developed based on split quaternion and octonion algebra, respectively. While SQTRU closely resembles QTRU, OTRU is designed as a multi-dimensional cryptosystem that utilizes octonion algebra, which is characterized by being both noncommutative and nonassociative. However, OTRU is significantly slower than NTRU at comparable dimensions, being 64 times slower for key generation and 8 to 16 times slower for encryption and decryption. Other interesting proposals based on quaternion algebra have emerged, including BQTRU [27], a noncommutative cryptosystem that utilizes the quaternion algebra of a bivariate ring of polynomials. BQTRU was claimed to be the fastest analog of NTRU, achieving key generation and encryption speeds that are more than two times faster than NTRU at equivalent security levels. However, a recent study by Raya et al. [28] disproved this claim, demonstrating that the lattice of BQTRU is overly structured and vulnerable to an extension of Gentry's attack [29]. The original proposal for Gentry's attack was introduced against typical NTRU when a composite value of n is used in the ring $\mathcal{R} = \mathbb{Z}[x]/(x^n - 1)$. Additionally, Ling and Mendelsohn [30] have theoretically introduced the NTRU in quaternion algebras, which can be proven to be IND-CPA as long as the discriminant of the quaternion is bounded.

Previous works have demonstrated the concept of noncommutative NTRU, but they did not provide a comprehensive implementation to establish the practicality of the

construction. DiTRU [31] is the first noncommutative NTRU to fully implement the cryptosystem, addressing all practical details, particularly inverting elements within the new noncommutative ring setting. However, the authors considered the one-layer Gentry attack, which results in the cryptosystem being twice as slow as standard NTRU when no decryption failures are allowed.

An improvement on DiTRU was introduced in [32, 33]. The authors in [33] generalized DiTRU to introduce a noncommutative variant from a semidirect product of cyclic groups over the ring of Eisenstein integers which we will refer to as Ei-TRU. This new construction features a faster key generation process as well as quicker encryption and decryption routines compared to DiTRU, making it almost as fast as NTRU when using the coefficients from the ring of Eisenstein integers. However, the same construction would be 1.5 times slower for encryption and decryption than typical NTRU over the integer ring while encrypting/decrypting messages of the same length. Table 1 summarizes the comparison of the discussed variants in the literature with the constructions proposed in this work.

1.2. Challenges and Motivation

As discussed in the previous section, challenges in proposing a noncommutative NTRU can be classified into the following points:

- **Theoretical designs and the absence of practical implementation:** Although several noncommutative NTRU proposals exist, most focus on theoretical aspects while neglecting crucial practical details related to cryptosystem design. A key concern lies in the key generation process, which requires inverting elements within the new noncommutative ring. This raises questions about how to implement the inversion efficiently, especially in constant time.
- **Insufficient security analysis:** Security analysis is a vital consideration for cryptographic designs. While some proposals introduce noncommutative variants, they alter the traditional NTRU design, creating new assumptions that require further scrutiny. For example, the first noncommutative NTRU variant proposed by Hoffstein and Silverman [19] was found to be vulnerable to an attack targeting the new design but not the standard NTRU design. Although the authors of BQTRU [27] conjectured that their structure would not be susceptible to Gentry’s attack, a study by Raya et al. [28] disproved this assertion, successfully breaking the moderate BQTRU parameter set that was said to achieve 2^{92} message security and 2^{166} key security, utilizing an extension of Gentry’s attack.
- **IND-CCA2 KEM based on loose security reductions:** The only two complete implementations of noncommutative NTRU, DiTRU [31] and Ei-TRU [33], have introduced Probabilistic Public

Key Encryption (PPKE) schemes. These encryption schemes can be converted into CCA2-secure implementations using the NAEP transformation [35], although they do not explicitly provide a KEM. The constructions follow the approach of NTRUencrypt, as presented in the first round of NIST submissions [10]. Consequently, the KEMs constructed on top of these public key schemes can be supported by only non-tight security proofs.

- **Incompetent efficiency compared to standard commutative NTRU over the ring of integers:** Introducing a noncommutative algebra may result in a more structured lattice, which could be susceptible to lattice attacks in smaller dimensions. Consequently, to achieve the same level of security as standard NTRU, it is necessary to increase the parameters, which also leads to a decrease in performance. For example, benchmarks of the only two fully implemented versions of noncommutative NTRU, DiTRU [31] and Ei-TRU [33], show that they are 2 and 1.5 times slower than NTRU over the ring of integers, respectively. Although [33] claims that performance is comparable to standard NTRU over the Eisenstein integers, similar improvements could also be achieved for standard NTRU in that context. In summary, none of the existing implementations of noncommutative NTRU demonstrate competent performance over the ring of integers compared to standard NTRU.

The previous challenges lead us to the following research questions:

Question 1

Is it possible to construct a noncommutative analog of NTRU while maintaining the performance of the cryptosystem over the ring of integers?

Question 2

Can a KEM be provided based on the new noncommutative structure from a Deterministic Public Key Encryption (DPKE) scheme rather than a PPKE with tighter security proofs?

1.3. Our Contribution

- **Efficient construction over the rings of integers and Eisenstein integers:** We provide two instantiations interpreted as group ring NTRU over the twisted dihedral group ring. While the first construction is built over the ring of integers, the other construction substitutes the integer ring with the ring of Eisenstein integers for faster computations. To our knowledge, our construction is the fastest over either commutative or noncommutative algebra as an NTRU-analogue scheme. Further, we propose an IND-CCA

Schemes	Noncommut.	Implementation	KEM	Rings	Note
NTRU HPS [34]	✗	✓	✓	$\frac{\mathbb{Z}[x]}{\langle x^N - 1 \rangle}, \frac{\mathbb{Z}[x]}{\langle \phi_N(x) \rangle}$	Proceeded to the final round of the NIST standardization process. We use it as a benchmark for performance comparison with other designs.
NTRU prime [12]	✗	✓	✓	$\frac{\mathbb{Z}[x]}{\langle x^p - x - 1 \rangle}$ (p : prime)	Progressed to the third round of the NIST standardization process.
BQTRU [27]	✓	✗	✗	quaternion algebra of bivariate polynomials	Fastest claimed variant of NTRU; however, recently broken by Raya et al. [28]
QTRU [24]	✓	✗	✗	quaternion algebra of univariate polynomials	4 times slower than NTRU.
SQTRU [25]	✓	✗	✗	split quaternions	16 times slower than NTRU.
DiTRU [31]	✓	✓	✗	$\mathbb{Z}D_N$	2 times slower than NTRU.
DiTRU over Eisensteins [32]	✓	✗	✗	$\mathbb{Z}[\omega]D_N$	1.5 times slower than NTRU.
NTRU from semidirect product [33]	✓	✓	✗	$\mathbb{Z}[\omega](C_N \rtimes C_3)$	Based on Eisenstein integers and 1.125 times slower than NTRU. However, 1.5 times slower than NTRU over the ring of integers.
This work	✓	✓	✓	$\mathbb{Z}D_N$ (twisted)	As efficient as NTRU over the ring of integers.
This work	✓	✓	✓	$\mathbb{Z}[\omega]D_N$ (twisted)	4/3 times faster than NTRU over the ring of Eisenstein integers.

Table 1: Various NTRU-like proposals in the literature.

KEM from the probabilistic PKE based on the ring of integers and the ring of Eisenstein integers.

- **A KEM derived from DPKE:** In contrast to the noncommutative proposals, such as DiTRU and Ei-TRU, which only provided an IND-CCA2 PPKE scheme, this work presents an IND-CCA2 KEM based on DPKE. This design is inspired by the NTRU-HRSS-KEM [11], which was included in the third-round submission for the NIST competition. This construction benefits from tighter security reductions.
- **Parameter selection and full-package implementation:** Considering the best-known attacks against NTRU-like constructions, we select the parameter sets for our designs targeting the security levels I, III, and V as per the security categories defined by NIST. Furthermore, we provide a reference implementation in C for the three proposed constructions as proof of our theoretical analysis. Our results show that the efficiency of our construction over the ring of integers can be further boosted using the ring of Eisenstein integers. The implementation is accessible at https://github.com/The-Isogeniest/KEMs_twisted_DiTRU.
- **Extensive comparison to existing constructions in literature:** We provide a detailed comparison with the NTRU proposals, focusing particularly on noncommutative constructions. To support this comparison, we have completed the KEMs implementations for constructions that originally only provided public key encryption schemes. Tables 2 and 3 present the memory and time comparisons, respectively. Table 2 compares the parameters and mem-

ory requirements of NTRU-HPS [34]¹, DiTRU [31], and Ei-TRU [33] with the constructions introduced in this work. The comparison results are organized into different categories based on the security levels achieved by the parameter sets. The column labeled “|poly|” refers to the length of the polynomial that represents both the private and public keys. The column “q” indicates the ciphertext modulus. The column “|CT|/|PK|” shows the size of the ciphertext/public key in bytes. The column “BKZ- β ” indicates the blocksize required for a lattice reduction attack to retrieve the private key. Meanwhile, the “(Sec.C, Sec.Q)” column records the security against the best classical and quantum attacks for the corresponding design. Lastly, the column “Dec. δ ” refers to the probability of decryption failure; for schemes designed to be deterministically correct with no decryption failures, this column indicates a value of 0. Table 3 presents a comparison of the time requirements². We report the CPU cycles needed for key generation, encapsulation, and decapsulation, along with ratios compared to the NTRU-HPS primitives as a reference point. The results indicate that our constructions offer the most compact and efficient designs within a noncommutative algebra and are also competitive with the original NTRU proposal in commutative algebra.

¹The parameter sets that achieve the security levels as per NIST guidelines have been revised in DiTRU [31]

²The comparisons presented in [31] and [36] focus on the time needed to encrypt and decrypt messages of the same length. In this work, however, we report the time required to encapsulate and decapsulate keys, as this is the standard measure for evaluating the performance of KEMs.

Table 2: Comparison of the parameters and the memory requirements for NTRU-like variants, including the revised NTRU-HPS [10] (commutative), DiTRU(noncommutative) [31], Ei-TRU(noncommutative) [33], and the constructions proposed in this work (noncommutative). The schemes are grouped into three categories according to their blocksize β and security estimation as per NIST’s security levels I, III, and V.

	 poly 	q	 sk 	 CT / PK 	BKZ-β	(Sec.C, Sec.Q)	Dec. δ
NTRU-HPS_587_2048	587	2048	118	808	456	(128, 120)	$< 2^{-128}$
DiTRU_541_2048	1082	2048	217	1488	445	(130, 118)	$< 2^{-128}$
Ei-TRU_109_701	327	701	131	818	464	(135, 122)	$< 2^{-128}$
Ei-TRU_127_2039	381	2039	153	1143	461	(134, 122)	0
DiTRU ⁺ _293_2048 [This work, Table 4]	586	2048	118	806	456	(128, 120)	$< 2^{-128}$
DiTRU ϕ^+ _317_2048 [This work, Table 5]	634	2048	1155	869	486	(133, 125)	0
E-DiTRU ⁺ _149_491 [This work, Table 7]	298	491	120	745	509	(131, 121)	$< 2^{-128}$
NTRU-HPS_863_2048	863	2048	173	1187	701	(192, 181)	$< 2^{-192}$
DiTRU_797_2048	1594	4096	319	2391	660	(193, 175)	$< 2^{-192}$
Ei-TRU_157_1013	471	1013	189	1296	663	(193, 175)	$< 2^{-192}$
Ei-TRU_181_2903	543	2903	218	1629	664	(193, 175)	0
DiTRU ⁺ _443_2048 [This work, Table 4]	886	2048	178	1219	723	(197, 186)	$< 2^{-192}$
DiTRU ⁺ _421_2048 [This work, Table 4]	842	2048	169	1158	682	(187, 177)	$< 2^{-192}$
DiTRU ϕ^+ _443_4096 [This work, Table 5]	886	4096	1712	1326	678	(194, 180)	0
DiTRU ϕ^+ _419_4096 [This wrok, Table 5]	838	4096	1622	1254	654	(182, 169)	0
E-DiTRU ⁺ _211_751 [This wrok, Table 7]	422	761	169	1055	733	(196, 183)	$< 2^{-192}$
NTRU-HPS_1109_4096	1109	4096	222	1664	893	(258, 237)	$< 2^{-256}$
DiTRU_1039_4096	2078	4096	416	3117	882	(258, 234)	$< 2^{-256}$
Ei-TRU_211_1361	633	1361	254	1741	886	(258, 234)	$< 2^{-256}$
Ei-TRU_241_3863	723	3863	290	2350	890	(259, 235)	$< 2^{-256}$
DiTRU ⁺ _557_4096 [This work, Table 4]	1114	4096	223	1671	898	(260, 238)	$< 2^{-256}$
DiTRU ⁺ _547_4096 [This work, Table 4]	1094	4096	219	1641	879	(253, 233)	$< 2^{-256}$
DiTRU ϕ^+ _587_4096 [This work, Table 5]	1174	4096	2260	1758	934	(265, 248)	0
DiTRU ϕ^+ _563_4096 [This work, Table 5]	1126	4096	2168	1686	891	(253, 236)	0
E-DiTRU ⁺ _263_1151 [This wrok, Table 7]	526	1151	211	1447	926	(257, 240)	$< 2^{-256}$

Table 3: Comparison of CPU cycles ($\times 10^3$) required for Key generation, Encapsulation, and Decapsulation of the key for NTRU-like variants, including the revised NTRU-HPS [10] (commutative), DiTRU (noncommutative) [31], Ei-TRU (noncommutative) [33], and the constructions proposed in this work (noncommutative). The schemes are grouped into three categories according to their blocksize β and security estimation as per NIST’s security levels I, III, and V.

	KeyGen	Encap	Decap	ratio
NTRU-HPS_587_2048	59 156	4 944	8 875	(1.00: 1.00: 1.00)
DiTRU_541_2048	78 612	14 983	28 220	(1.32: 3.03: 3.18)
Ei-TRU_109_701	26 909	4 954	8 485	(0.45: 1.00: 0.96)
Ei-TRU_127_2039	36 080	6 342	11 037	(0.61: 1.28: 1.24)
DiTRU ⁺ _293_2048 [This work, Table 4]	24 118	4 911	8 817	(0.41: 0.99: 0.99)
DiTRU ϕ^+ _317_2048 [This work, Table 5]	61 553	5 470	13 757	(1.04: 1.11: 1.55)
E-DiTRU ⁺ _149_491 [This work, Table 7]	36 813	3 725	6 737	(0.62: 0.75: 0.76)
NTRU-HPS_863_2048	1 38 083	10 029	18 674	(1.00: 1.00: 1.00)
DiTRU_797_4096	1 72 496	31 469	60 078	(1.25: 3.14: 3.22)
Ei-TRU_157_1013	53 739	9 055	16 039	(0.39: 0.90: 0.86)
Ei-TRU_181_2903	70 790	11 481	20 612	(0.51: 1.14: 1.10)
DiTRU ⁺ _443_2048 [This work, Table 4]	56 928	10 477	19 517	(0.41: 1.04: 1.05)
DiTRU ⁺ _421_2048 [This work, Table 4]	49 433	9 512	17 727	(0.36: 0.95: 0.95)
DiTRU ϕ^+ _443_4096 [This work, Table 5]	1 30 671	10 472	27 197	(0.95: 1.04: 1.46)
DiTRU ϕ^+ _419_4096 [This work, Table 5]	1 16 143	9 247	23 822	(0.84: 0.92: 1.28)
E-DiTRU ⁺ _211_751 [This work, Table 7]	73 269	6 779	12 572	(0.53: 0.68: 0.67)
NTRU-HPS_1109_4096	2 16 990	15 870	29 913	(1.00: 1.00: 1.00)
DiTRU_1039_4096	2 76 955	52 249	1 00 841	(1.27: 3.29: 3.37)
Ei-TRU_211_1361	96 013	14 951	27 229	(0.44: 0.94: 0.91)
Ei-TRU_241_3863	1 24 281	18 951	35 052	(0.57: 1.19: 1.17)
DiTRU ⁺ _557_4096 [This work, Table 4]	86 721	16 053	30 274	(0.40: 1.01: 1.01)
DiTRU ⁺ _547_4096 [This work, Table 4]	83 628	15 520	29 225	(0.39: 0.98: 0.98)
DiTRU ϕ^+ _587_4096 [This work, Table 5]	2 21 588	17 577	47 041	(1.02: 1.11: 1.57)
DiTRU ϕ^+ _563_4096 [This work, Table 5]	2 04 758	16 111	43 064	(0.94: 1.02: 1.44)
E-DiTRU ⁺ _263_1151 [This work, Table 7]	1 11 824	10 049	18 685	(0.52: 0.63: 0.62)

2. Preliminaries

2.1. Notations

Symbol	Description
$\mathbb{Z}$	set of integers
$\mathbb{Z}_q$	ring of integers modulo a positive integer q
$\mathbb{R}$	set of real numbers
$\mathbb{R}^n$	Euclidean space of dimension n
$\mathbb{C}$	set of complex numbers
$\mathbf{v}$	vectors are represented by bold lowercase letters
$\mathbf{B}$	matrices are represented by bold uppercase letters
$\ \cdot\ $	for a vector $\mathbf{v} = (v_1, v_2, \dots, v_n) \in \mathbb{R}^n$, $\ \mathbf{v}\ $ is the Euclidean (ℓ_2 norm) calculated as $\ \mathbf{v}\ = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$.
$*$	multiplication between two elements with respect to the underlying algebraic structure over which they are defined

2.2. Definitions

Definition 1 (Lattice). A lattice $\mathcal{L}(\mathbf{B})$ generated by the linearly independent rows $\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1} \in \mathbb{R}^d$ of the generator matrix $\mathbf{B} \in \mathbb{R}^{n \times d}$ is the span of all the integral linear combinations of the rows of $\mathbf{B}$, i.e.,

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=0}^{n-1} \alpha_i \mathbf{b}_i : \alpha_i \in \mathbb{Z} \right\}. \quad (1)$$

Here, n is the rank and d is the dimension of the lattice $\mathcal{L}(\mathbf{B})$. For $n = d$, we call the lattice a full-rank lattice, and when $\mathbf{b}_i \in \mathbb{Z}^d$, we refer to the lattice as an integral lattice. In the scope of this paper, we are dealing with full-rank integral lattices unless it is stated otherwise. A d -dimensional lattice $\mathcal{L}(\mathbf{B})$ is called q -ary, if $q\mathbb{Z}^d \subset \mathcal{L}(\mathbf{B})$ for some positive integer q . The volume or the determinant of the lattice is calculated as $\det(\mathcal{L}(\mathbf{B})) = \sqrt{|\det(\mathbf{B} * \mathbf{B}^T)|}$. For a basis $\mathbf{B} = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{d-1}\}$, the Gram-Schmidt orthogonalized basis is denoted by $\{\mathbf{b}_0^*, \mathbf{b}_1^*, \dots, \mathbf{b}_{d-1}^*\}$ and is defined as

$$\mathbf{b}_i^* = \pi_i(\mathbf{b}_i) = \mathbf{b}_i - \sum_{j=0}^{i-1} \frac{\mathbf{b}_i \cdot \mathbf{b}_j^*}{\|\mathbf{b}_j^*\|^2} \mathbf{b}_j^*,$$

where π_i is the projection on the space orthogonal to the span of $\{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{i-1}\}$.

Definition 2 (SVP). Let $\mathcal{L}(\mathbf{B}) \subset \mathbb{R}^d$ be a lattice generated by $\mathbf{B}$, then the Shortest Vector Problem (SVP) involves finding the shortest nonzero vector $\mathbf{v} \in \mathcal{L}(\mathbf{B}) - \{\mathbf{0}\}$. The length of the $\mathbf{v}$ is denoted as $\lambda_1(\mathcal{L}(\mathbf{B}))$.

Definition 3 (Gaussian heuristic). Let $\mathcal{L}(\mathbf{B}) \subset \mathbb{R}^d$ be a full-rank lattice of dimension d , then the Gaussian heuristic, denoted as $gh(\mathcal{L}(\mathbf{B}))$, estimates the norm of the shortest vector for large d as

$$gh(\mathcal{L}(\mathbf{B})) = \sqrt{d/2\pi e} \cdot \det(\mathcal{L}(\mathbf{B}))^{1/d}. \quad (2)$$

2.3. Public Key Encryption Scheme (PKE)

Before introducing NTRU, we give a few definitions related to the PKE scheme. Let $\mathcal{M}$ indicate the message space, $\mathcal{K}$ indicate the key space, and $\mathcal{C}$ indicate the ciphertext space, then a PKE is defined as a triple of Probabilistic Polynomial Time (PPT) algorithms (KeyGen, Enc, Dec), where:

- **KeyGen**(1^λ): given a determined level of security λ , it generates the pair (pk, sk) , where pk indicates the public key and sk indicates the secret key. We denote this process as $(pk, sk) \leftarrow^{\$} \text{KeyGen}(1^\lambda)$.
- **Enc**(m, pk, r): For an input message $m \in \mathcal{M}$ and possibly a random string r , the encryption routine encrypts the message m using the public key pk and produces the ciphertext c which belongs to the ciphertext space $\mathcal{C}$. We denote this process as $c = \text{Enc}(m, pk, r)$.
- **Dec**(c, sk): For input c as the ciphertext, the decryption routine decrypts c and returns the decrypted message $m' \in \mathcal{M}$ or a symbol $\perp \notin \mathcal{M}$ indicating that $c \notin \mathcal{C}$. We denote this process as $m = \text{Dec}(c, sk)$.

We say that a PKE is correct if $\text{Dec}(\text{Enc}(m, pk), sk) = m$ for all $m \in \mathcal{M}$, and we call a PKE δ -correct if the probability of $\text{Dec}(\text{Enc}(m, pk), sk) \neq m$ is at most δ in the worst case. Further, we say that a PKE is γ -spread [37, Definition F3] if any $m \in \mathcal{M}$ is mapped to $c \in \mathcal{C}$ with a probability bounded above by $2^{-\gamma}$, where the probability is taken considering all the key pairs and the randomness in the encryption process, i.e.,

$$\min_{\substack{m \in \mathcal{M} \\ (sk, pk)}} \left(-\log \max_{c \in \mathcal{C}} \Pr[c = \text{Enc}(m, pk, r)] \right) \geq \gamma.$$

In this context, we further recall OW-CPA and IND-CPA security notions to describe a PKE.

Definition 4 (OW-CPA PKE). A PKE scheme is one-way secure under the chosen plaintext attack, denoted as OW-CPA PKE, if a PPT adversary $\mathcal{A}$ has a negligible advantage, defined as $\text{Adv}_{PKE}^{\text{OW-CPA}} := \Pr[m' = m^*]$, in the security game outlined in Figure 1, for the required security determined by the parameter λ . In other words, for any ciphertext $c^* = \text{Enc}(m^*, pk)$ generated by the challenger, a PPT adversary can reconstruct the message m^* with only a negligible probability.

Definition 5 (IND-CPA PKE). A PKE scheme is indistinguishable under the chosen plaintext attack, denoted as IND-CPA PKE, if a PPT adversary $\mathcal{A}$ has a negligible advantage, defined as $\text{Adv}_{PKE}^{\text{IND-CPA}} := |\Pr[b = b'] - \frac{1}{2}|$, in the security game outlined in Figure 1, for the required security determined by the parameter λ . In other words, the adversary can select two messages m_0, m_1 of his choice and send them to the challenger; the challenger selects a random $b \in \{0, 1\}$ and computes $c = \text{Enc}(m_b, pk)$, the adversary can distinguish which message has been encrypted with only a negligible advantage.

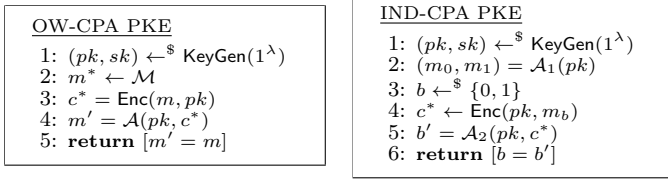


Figure 1: Security games for OW-CPA and IND-CPA.

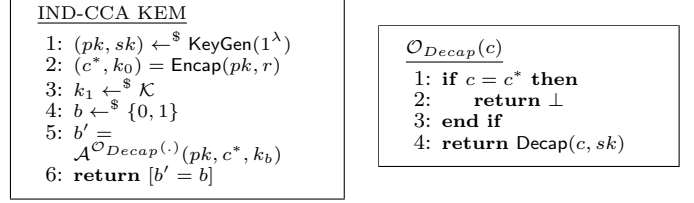


Figure 2: Security game for IND-CCA KEM.

2.4. Key Encapsulation Mechanism

A Key Encapsulation Mechanism or KEM establishes a shared secret key between two parties over an insecure channel. As in the PKE, a KEM is defined as a triple of PPT algorithms (KeyGen , Encap , Decap), where

1. $\text{KeyGen}(1^\lambda)$: on a determined level of security λ , it generates the pair (pk, sk) , where pk indicates the public key and sk indicates the private key. We denote this process as $(pk, sk) \xleftarrow{\$} \text{KeyGen}(1^\lambda)$.
2. $\text{Encap}(pk, r)$: For an input pk and possibly a random string r , the encapsulation routine generates a secret session key $k \in \mathcal{K}$ and a ciphertext $c \in \mathcal{C}$ corresponding to the secret key k . We denote this process as $(k, c) = \text{Encap}(pk, r)$.
3. $\text{Decap}(c, sk)$: For an input $c \in \mathcal{C}$ and a secret key sk , the decapsulation routine returns the secret session key k corresponding to the ciphertext c . We denote this process as $k = \text{Decap}(c, sk)$.

We call a KEM δ -correct if the probability of $\text{Decap}(c, sk) \neq k$ while $(k, c) = \text{Encap}(pk, r)$ is at most δ .

Definition 6 (IND-CCA KEM). *A KEM is indistinguishable under the chosen ciphertext attack, denoted as IND-CCA KEM, if a PPT adversary $\mathcal{A}$ has a negligible advantage, defined as $\text{Adv}_{\text{KEM}}^{\text{IND-CCA}} := |\Pr[b = b'] - \frac{1}{2}|$, in the security game outlined in Figure 2, considering the security parameter λ . In other words, the challenger shares with the adversary (k_0, k_1, c^*) , where k_0 is the genuine key, k_1 is a randomly sampled key from the key space $\mathcal{K}$, and c^* is the ciphertext related to the genuine key; the adversary can distinguish which key is related to c^* with only a negligible advantage, even if it has access to a decapsulation oracle that decapsulates all queries on $c \in \mathcal{C}$ given that $c \neq c^*$.*

2.4.1. The Fujisaki-Okamoto (FO) transform.

FO [38, 39] is a general transformation that converts any PKE scheme secure under chosen-plaintext attacks (CPA-secure) into a KEM that is secure under chosen-ciphertext attacks (CCA-secure). Starting from an OW-CPA PKE or an IND-CPA PKE, one can construct an IND-CCA KEM using the FO transformation. Figure 3 outlines the steps of this transformation along with references to the

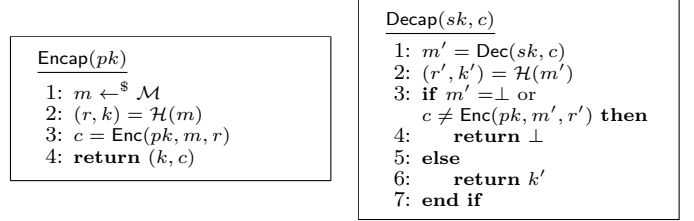


Figure 3: The Fujisaki-Okamoto general transformation.

theorems that describe how to convert a given PKE into CCA-KEM in Figure 4.

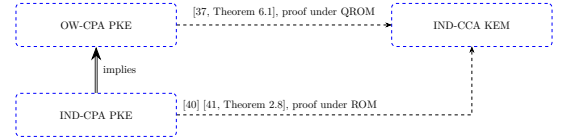


Figure 4: Transforming an OW-CPA/IND-CPA into an IND CCA-KEM under proofs in the Random Oracle Model/Quantum Random Oracle Model.

2.5. NTRU

We refer to the initial variant of NTRU [7] as typical or standard NTRU, and give a simplified description of the cryptosystem as mentioned in [42].

2.5.1. Set up.

- The tuple (N, p, q, d) represents the public parameters with N, p being prime integers with $\gcd(p, q) = \gcd(N, q) = 1$, $p \ll q$, $d \leq \lfloor N/3 \rfloor$, and $q > (6d + 1)p$.
- Let $\mathcal{R}, \mathcal{R}_p$, and $\mathcal{R}_q$ be the convolutional rings defined as

$$\mathcal{R} = \frac{\mathbb{Z}[x]}{(x^N - 1)}, \quad \mathcal{R}_p = \frac{\mathbb{Z}_p[x]}{(x^N - 1)}, \quad \mathcal{R}_q = \frac{\mathbb{Z}_q[x]}{(x^N - 1)}. \quad (3)$$

- Let $\mathcal{T}_N(d_1, d_2)$ be the set of all ternary polynomials of maximum degree $N - 1$ with d_1 coefficients equal to $+1$, d_2 coefficients equal to -1 , and rest are 0.
- Let $\mathcal{T}_N$ be the set of all ternary polynomials with maximum degree $N - 1$.

- Upon setting a space and a *seed*, the procedure **Sampler** samples a polynomial from the specified space according to the input *seed*.
- Let $f(x) \in \mathcal{R}_q$ then the applying the procedure **Centerlift**(f, q) outputs the unique polynomial $f'(x) \in \mathcal{R}$ with coefficients in the range $(-q/2, q/2]$ such that $f'(x) \pmod{q} = f(x)$.

2.5.2. Sketched cryptosystem.

A simplified description of NTRU is given in Figure 5 where $\mathcal{L}_f = \mathcal{T}(d+1, d)$, $\mathcal{L}_g = \mathcal{T}(d, d)$, $\mathcal{L}_r = \mathcal{T}(d, d)$, and $\mathcal{L}_m = \mathcal{T}_N$.

The NTRU instance, the decision NTRU problem, and the search NTRU problem are defined as follows:

Definition 7 (NTRU instance). *For $\eta > 0$ (real) and $q \geq 2$ (integer), an NTRU instance associated with (η, q) is an element $h \in \mathcal{R}_q$ calculated as $f^{-1} * g \pmod{q}$ such that $(f, g) \neq (\mathbf{0}, \mathbf{0}) \in \mathcal{R}^2$ and $\|f\|, \|g\| \leq \sqrt{q}/\eta$.*

Definition 8 (Decision NTRU problem). *For $\eta > 0$ (real), $q \geq 2$ (integer), and certain distribution $\mathcal{D}$, the decisional NTRU problem asks to distinguish between $h \in \mathcal{D}$ calculated as $h = f^{-1} * g \pmod{q}$ and a random element u sampled uniformly from the ring $\mathcal{R}_q$. The advantage of an adversary against the decisional NTRU problem must be negligible and is defined as*

$$\text{Adv}_{(\mathcal{D}, \eta, q)}^{D\text{-NTRU}} = \left| \Pr_{h \leftarrow \mathcal{D}}(\mathcal{A}(h) = 1) - \Pr_{u \leftarrow U(\mathcal{R}_q)}(\mathcal{A}(u) = 1) \right|.$$

Definition 9 (Search NTRU problem). *For η, η' (real) with $\eta \geq \eta' > 0$, $q \geq 2$ (integer) and a certain distribution $\mathcal{D}$, given h from the distribution $\mathcal{D}$, the NTRU search problem (average case) asks for finding $(f, g) \in \mathcal{R}^2$ such that $f * h = g \pmod{q}$ and $\|f\|, \|g\| \leq \sqrt{q}/\eta'$. The advantage of an adversary against the search NTRU problem must be negligible and is defined as*

$$\text{Adv}_{(\mathcal{D}, \eta, \eta', q)}^{S\text{-NTRU}} = \Pr_{h \leftarrow \mathcal{D}} \left(\mathcal{A}(h) = (f, g) \neq (\mathbf{0}, \mathbf{0}) \left| \begin{array}{l} f * h = g \pmod{q}, \\ \|f\|, \|g\| \leq \frac{\sqrt{q}}{\eta'} \end{array} \right. \right).$$

2.5.3. NTRU as a lattice-based cryptosystem

To attack the cryptosystem, the attacker needs to solve the NTRU search problem, i.e., finding f', g' : two polynomials with small coefficients that satisfy the NTRU equation $f' * h = g' \pmod{q}$. To achieve this, the attacker can either apply search techniques to identify possible polynomials with small coefficients from the underlying ring or use lattice reduction attacks to find a short vector that

corresponds to a valid solution. The most straightforward way is to build the lattice

$$\mathcal{L}_{CS} = \{(\mathbf{f}, \mathbf{g}) \in \mathbb{Z}^{2N} \mid f * h = g \pmod{q}\},$$

that was introduced firstly by Coppersmith and Shamir [20]. An attacker, knowing the public key h , can build the basis $\mathcal{B}_{CS}$ for the lattice as

$$\mathcal{B}_{CS} = \begin{pmatrix} \mathbf{I}_N & \mathbf{H} \\ \mathbf{0}_N & q\mathbf{I}_N \end{pmatrix},$$

where the matrix $\mathbf{H}$ is a right circulant matrix when the modulus $x^N - 1$ is used, i.e.,

$$\mathbf{H} = \begin{pmatrix} h_0 & h_1 & \dots & h_{N-1} \\ h_{N-1} & h_0 & \dots & h_{N-2} \\ \vdots & \vdots & \ddots & \vdots \\ h_1 & h_2 & \dots & h_0 \end{pmatrix}. \quad (4)$$

Since the secret key (f, g) corresponds to a ternary polynomial and satisfies the NTRU equation $f * h = g \pmod{q}$, the coefficient vector $(\mathbf{f}, \mathbf{g})$ is, with high probability, the shortest vector in the lattice $\mathcal{L}_{CS}$. Consequently, an attacker can potentially retrieve a valid decryption key by reducing the basis. Moreover, it is important to note that not only does the vector $(\mathbf{f}, \mathbf{g})$ lie in the lattice $\mathcal{L}_{CS}$, but so do all its rotations with respect to the underlying algebra. That is all vectors corresponding to the polynomials $(x^i * f, x^i * g)$ for all $0 \leq i \leq N-1$ are also contained within $\mathcal{L}_{CS}$.

2.5.4. Gentry's attack.

Different rings have been introduced into the original design of NTRU with the motivation of providing faster computation or reducing the attack surface. In the former case, NTRU is mainly defined over NTT-friendly rings where the multiplication can be boosted, and for that, the degree N of the polynomial modulus is recommended to be a power of 2.

The first attempt to define such a variant of NTRU [19] was unsuccessful and found to be vulnerable to an algebraic attack introduced by Gentry [29]. Gentry's attack idea is related to the Chinese Remainder Theorem (CRT) for rings.

Theorem 1 (Chinese Remainder Theorem for ring). [43, Theorem 17, Section 7.6] *Let $\mathcal{I}_1, \mathcal{I}_2, \dots, \mathcal{I}_k$ be pairwise coprime ideals of a ring $\mathcal{A}$, and let $\mathcal{I} = \cap \mathcal{I}_j$. Then the ring $\mathcal{A}/\mathcal{I}$ is isomorphic to the direct product $\frac{\mathcal{A}}{\mathcal{I}_1} \times \frac{\mathcal{A}}{\mathcal{I}_2} \times \dots \times \frac{\mathcal{A}}{\mathcal{I}_k}$. Consequently, any element $r \in \frac{\mathcal{A}}{\mathcal{I}}$ is mapped according to the map*

$$r \rightarrow (r \pmod{\mathcal{I}_1}, r \pmod{\mathcal{I}_2}, \dots, r \pmod{\mathcal{I}_k}),$$

and this map can be inverted efficiently.

For the ring $\mathcal{R}_q = \frac{\mathbb{Z}_q[x]}{(x^N-1)}$ ($N = 2^n$), Gentry defines the following isomorphism

$$\frac{\mathbb{Z}_q[x]}{(x^N-1)} \cong \frac{\mathbb{Z}_q[x]}{(x^{N/2}-1)} \times \frac{\mathbb{Z}_q[x]}{(x^{N/2}+1)}. \quad (5)$$

$\text{KeyGen}(1^\lambda)$	$\text{Encrypt}(h, m, \text{seed}_r)$	$\text{Decrypt}((f, f_p), c)$
1: initialize Sampler with $\mathcal{L}_f$ and seed_f	1: initialize Sampler with $\mathcal{L}_r$ and seed_r	1: $t \leftarrow f * c \pmod{q}$
2: do $f \leftarrow \text{Sampler}$ until f is invertible in R_q and R_p	2: $r \leftarrow \text{Sampler}$	2: $a \leftarrow \text{Centerlift}(t, q)$
3: Let f_q and f_p be the inverses of f in R_q and R_p , respectively	3: $c = pr * h + m \pmod{q}$	3: $m' \leftarrow f_p * a \pmod{p}$
4: initialize Sampler with $\mathcal{L}_g$ and seed_g	4: return c	4: $m \leftarrow \text{Centerlift}(m', p)$
5: $g \leftarrow \text{Sampler}$		5: return m
6: $h \leftarrow g * f_q \pmod{q}$		
7: return $((f, f_p), h)$		

Figure 5: A sketched description of the initial NTRU HPS cryptosystem, as introduced in [42].

It can be checked that any polynomial $r(x) = r_0(x) + x^{N/2} * r_1(x) \in \mathcal{R}_q$, where $r_i(x)$ is a polynomial of maximum degree $N/2 - 1$, is mapped according to the isomorphism as

$$r(x) \rightarrow (r_0(x) + r_1(x), r_0(x) - r_1(x)).$$

This map sends the ternary polynomials $f, g \in \mathcal{R}_q$ to polynomials with small coefficients in the set $\{-2, -1, 0, 1, 2\}$. In addition, the NTRU equation holds, i.e.

$$h_0 + h_1 = (f_0 + f_1)^{-1} * (g_0 + g_1) \in \frac{\mathbb{Z}_q[x]}{(X^{N/2} - 1)},$$

$$h_0 - h_1 = (f_0 - f_1)^{-1} * (g_0 - g_1) \in \frac{\mathbb{Z}_q[x]}{(X^{N/2} + 1)}.$$

Consequently, the attacker can build the two lattices $\mathcal{L}_{CS}^+$ and $\mathcal{L}_{CS}^-$ generated from the bases

$$\mathcal{B}_{CS}^+ = \begin{pmatrix} \mathbf{I}_{N/2} & \mathbf{H}_0 + \mathbf{H}_1 \\ \mathbf{0}_{N/2} & q\mathbf{I}_{N/2} \end{pmatrix},$$

$$\mathcal{B}_{CS}^- = \begin{pmatrix} \mathbf{I}_{N/2} & \mathbf{H}_0 - \mathbf{H}_1 \\ \mathbf{0}_{N/2} & q\mathbf{I}_{N/2} \end{pmatrix},$$

respectively and reduce them to get the vectors $(\mathbf{f}_0 + \mathbf{f}_1, \mathbf{g}_0 + \mathbf{g}_1) \in \mathcal{L}_{CS}^+$ and $(\mathbf{f}_0 - \mathbf{f}_1, \mathbf{g}_0 - \mathbf{g}_1) \in \mathcal{L}_{CS}^-$ that will eventually allow retrieving the vector $(\mathbf{f}, \mathbf{g}) \in \mathcal{L}_{CS}$. Halving the dimension of the lattice makes the problem of getting the secret key much easier for a lattice reduction algorithm. The same idea can be exploited to further reduce the lattice dimensions for particular parameters as discussed in Gentry's original paper, and hence the ring $\mathbb{Z}_q[x]/(x^N - 1)$ for $N = 2^n$ is not considered a secure design as solving the Shortest Vector Problem becomes easier.

Interestingly, using N as a power of 2 in the NTT-friendly ring $\mathcal{R}_q^+ = \mathbb{Z}_q[x]/(x^N + 1)$ does not lead to such an attack. The ring $\mathcal{R}_q^+$ has been widely used in designing lattice-based schemes. Falcon [8], the selected signature scheme for standardization by NIST, is constructed over the ring $\mathcal{R}_q^+$ with security based on the NTRU assumption, assuming that the selection of this ring does not lead to any similar attack that reduces the lattice dimension. Similarly, Kyber [44], the selected KEM by NIST based on the LWE assumption, is designed over the same ring. The authors of Kyber [44, Page 35] state that "The polynomial

$x^N + 1$ does not have any factor (modulo q) with small degree and norm. For example, all the factors of degree $n/2$ have at least ℓ_2 norm of $\sqrt{q}$." It implies that a short polynomial in $\mathcal{R}_q^+$ cannot be mapped to a short polynomial in smaller dimensional subrings, and therefore, the attacker cannot benefit from a Gentry-like attack which was originally introduced for the ring $\mathcal{R}_q$. In Section 3, we provide another perspective to look at the dimension-reduction that supports this conjecture in favor of the ring $\mathcal{R}_q^+$. In this paper, we refer to the construction of NTRU over the ring $\mathcal{R}_q^+$ as NTRU⁺.

3. A perspective for Generalization

Yasuda et al. [45] proposed a general concept for defining NTRU variants based on group rings, referred to as GR-NTRU. Before introducing GR-NTRU, we will first provide the necessary definitions of group rings.

Definition 10 (Group ring). *Let R be a ring and $G = \{g_1, g_2, \dots, g_n\}$ be a finite group of order n . Then, the group ring of G over R is*

$$RG = \left\{ r = \sum_{i=1}^n r_{g_i} g_i : r_{g_i} \in R \text{ for } i = 1, 2, \dots, n \right\}. \quad (6)$$

$(RG, +, *)$ forms a ring where $+$ indicates the sum of two elements from RG and $*$ refers to the multiplication of them. Let $r = \sum_{i=1}^n r_{g_i} g_i, s = \sum_{i=1}^n s_{g_i} g_i \in RG$. Then

1. $r + s = \sum_{i=1}^n (r_{g_i} + s_{g_i}) g_i$.
2. $r * s = \sum_{i=1}^n \left(\sum_{g_h g_k = g_i} r_{g_h} s_{g_k} \right) g_i$.

For an element $r = \sum_{i=1}^n r_{g_i} g_i \in RG$, the coefficient vector of r is written as $\mathbf{r} = (r_{g_1}, r_{g_2}, \dots, r_{g_n}) \in R^n$. Furthermore, any element of the ring RG can be represented as a matrix [46] as follows

$$\mathbf{M}_{RG}(r) = \begin{pmatrix} r_{g_1^{-1}g_1} & r_{g_1^{-1}g_2} & \dots & r_{g_1^{-1}g_n} \\ r_{g_2^{-1}g_1} & r_{g_2^{-1}g_2} & \dots & r_{g_2^{-1}g_n} \\ \vdots & \vdots & \ddots & \vdots \\ r_{g_n^{-1}g_1} & r_{g_n^{-1}g_2} & \dots & r_{g_n^{-1}g_n} \end{pmatrix}. \quad (7)$$

The map τ that sends an element $r \in RG$ to the matrix $\mathbf{M}_{RG}(r)$ is a ring homomorphism [46, Theorem 1], i.e., for any two elements $r, s \in RG$, it satisfies

1. $\tau(r + s) = \tau(r) + \tau(s) = \mathbf{M}_{RG}(r) + \mathbf{M}_{RG}(s)$.
2. $\tau(r * s) = \tau(r) * \tau(s) = \mathbf{M}_{RG}(r) * \mathbf{M}_{RG}(s)$.

3.1. GR-NTRU

Another perspective on the standard NTRU defined over $\mathbb{Z}_q[x]/(x^N - 1)$ is to view it as a group ring, where G is the cyclic group $C_N = \langle x \mid x^N = 1 \rangle$ and R is the ring $\mathbb{Z}$. One can verify that the matrix $\mathbf{M}_{RG}(r)$ for any element $r \in RG = \mathbb{Z}C_N$ is a right circulant matrix as given earlier in Equation 4. In this context, the construction of NTRU can be generalized to other group rings where the group G does not need to be a commutative group. Therefore, GR-NTRU can be defined exactly as in Figure 5 except that the elements are being sampled from RG , $\frac{R}{\langle q \rangle}G$, and $\frac{R}{\langle p \rangle}G$ instead of $\mathcal{R}$, $\mathcal{R}_q$, and $\mathcal{R}_p$, respectively. Figure 6 illustrates that standard NTRU is a specific instance of GR-NTRU.

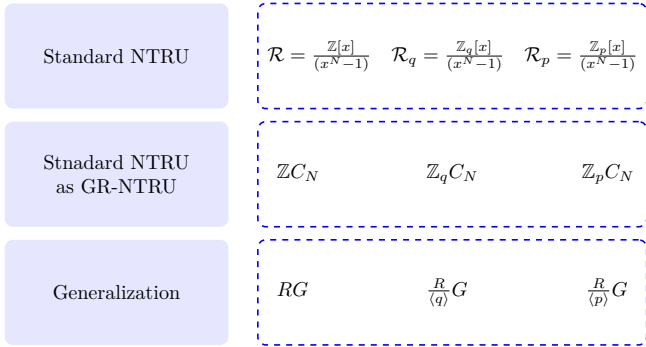


Figure 6: GR-NTRU as a framework to generalize NTRU variants. The illustration demonstrates that the standard NTRU can be viewed as a GR-NTRU based on cyclic groups and the ring of integers. This concept can also be extended to other group rings.

It is observed that the ring $\mathcal{R}_q^+$ cannot be directly defined within the GR-NTRU framework since the multiplication is *twisted*, i.e., $x^N = -1$. This leads us to introduce a more generalized structure known as twisted GR-NTRU.

3.2. Twisted GR-NTRU

Definition 11 (2-cocycle). A function $\chi : G \times G \rightarrow R^*$ is called a 2-cocycle if it satisfies the following two properties:

- $\chi(1, 1) = 1$, and
- for all $g_1, g_2, g_3 \in G$, the cocycle condition

$$\chi(g_1 g_2, g_3) \chi(g_1, g_2) = \chi(g_1, g_2 g_3) \chi(g_2, g_3) \quad (8)$$

holds.

Definition 12 (Twisted group ring). $R^\chi G$ denotes the twisted group ring constructed from the group G and the ring R according to the rules of the 2-cocycle χ . The ring

$R^\chi G$ adheres to Definition 10, with the exception that its multiplication is defined by

$$\sum_{i=1}^n r_{g_i} g_i * \sum_{i=1}^n s_{g_i} g_i = \sum_{i=1}^n \left(\sum_{g_h g_k = g_i} r_{g_h} s_{g_k} \chi(g_h, g_k) \right) g_i. \quad (9)$$

Further, any $r = \sum_{i=1}^n r_{g_i} g_i \in R^\chi G$ can be represented as

$$\mathbf{M}_{RG}^\chi(r) = \begin{pmatrix} \chi(g_1, g_1^{-1} g_1) r_{g_1^{-1} g_1} & \chi(g_1, g_1^{-1} g_2) r_{g_1^{-1} g_2} & \cdots & \chi(g_1, g_1^{-1} g_n) r_{g_1^{-1} g_n} \\ \chi(g_2, g_2^{-1} g_1) r_{g_2^{-1} g_1} & \chi(g_2, g_2^{-1} g_2) r_{g_2^{-1} g_2} & \cdots & \chi(g_2, g_2^{-1} g_n) r_{g_2^{-1} g_n} \\ \vdots & \vdots & \ddots & \vdots \\ \chi(g_n, g_n^{-1} g_1) r_{g_n^{-1} g_1} & \chi(g_n, g_n^{-1} g_2) r_{g_n^{-1} g_2} & \cdots & \chi(g_n, g_n^{-1} g_n) r_{g_n^{-1} g_n} \end{pmatrix}. \quad (10)$$

It is easy to observe that a group ring is simply a twisted group ring with $\chi = 1$. Consequently, a twisted GR-NTRU cryptosystem is defined similarly to a GR-NTRU with multiplication twisted according to the accompanying 2-cocycle rules.

As a result, the NTRU problem can be generalized over different rings. In this sense, the generalized decision and search NTRU problems are defined exactly as in Definitions 8 and 9, respectively, where h is sampled from the underlying group ring $\frac{R}{\langle q \rangle}G$ instead of the standard NTRU ring $\mathcal{R}_q$.

3.2.1. Generalized Coppersmith Shamir lattice

As the cryptosystem design based on GR-NTRU or twisted GR-NTRU does not change, the hard assumption of NTRU also remains intact. The only difference is that private polynomials are sampled from the new ring RG instead of the truncated polynomial ring as in the standard NTRU. Therefore, finding the private key is still related to the SVP in a lattice constructed from the public key h . The lattice structure changes based on the underlying ring and is generated from the basis $\mathcal{B}_{CS}^{Gen}$ as described in equation 11.

$$\mathcal{B}_{CS}^{Gen} = \begin{pmatrix} \mathbf{I}_{|G|} & \mathbf{M}_{RG}^\chi(h) \\ \mathbf{0}_{|G|} & q \mathbf{I}_{|G|} \end{pmatrix}. \quad (11)$$

In this context, we rebuild the lattice of the NTRU when it is constructed over the ring $\mathcal{R}_q$ and $\mathcal{R}_q^+$, respectively, and the polynomial modulus degree N is a power of 2. In the former case, we can observe that NTRU is an instance of GR-NTRU, and the associated lattice is generated from the basis $\mathcal{B}_{CS}^{NTRU}$ given by

$$\mathcal{B}_{CS}^{NTRU} = \begin{pmatrix} \mathbf{I}_{N/2} & \mathbf{0}_{N/2} & \mathbf{H}_0 & \mathbf{H}_1 \\ \mathbf{0}_{N/2} & \mathbf{I}_{N/2} & \mathbf{H}_1 & \mathbf{H}_0 \\ \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & q \mathbf{I}_{N/2} & \mathbf{0}_{N/2} \\ \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & q \mathbf{I}_{N/2} \end{pmatrix}, \quad (12)$$

where $\mathbf{H}_i$ is an $(N/2 \times N/2)$ matrix. It is not difficult to see that one can define a homomorphism, as illustrated in Figure 7, which simplifies the problem of retrieving the

short vector $(\mathbf{f}, \mathbf{g}) \in \mathcal{L}_{CS}^{NTRU}$. This simplification arises from the nice symmetry present in the upper right part of the matrix $\mathcal{B}_{CS}^{NTRU}$. The homomorphisms defined on the matrices can be viewed as a different interpretation of Gentry's attack, as discussed earlier in Section 2.

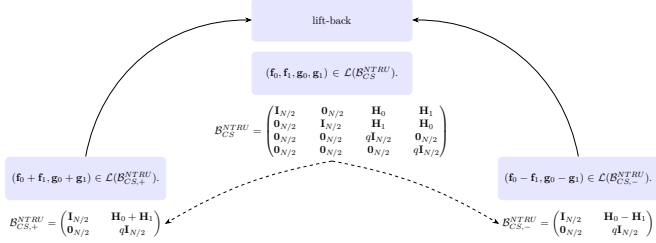


Figure 7: A dimension reduction homomorphism reduces the difficulty of solving the Shortest Vector Problem (SVP) in a lattice of dimension $2N$ to the problem of solving the SVP in two lattices of dimension N . This applies when N is a power of 2, and the modulus $x^N - 1$ is used to construct the NTRU ring.

On the other hand, the lattice associated with the NTRU variant constructed over the ring $\mathcal{R}_q^+$ is generated from the basis $\mathcal{B}_{CS}^{NTRU+}$ given by

$$\mathcal{B}_{CS}^{NTRU+} = \begin{pmatrix} \mathbf{I}_{N/2} & \mathbf{0}_{N/2} & \mathbf{H}_0 & \mathbf{H}_1 \\ \mathbf{0}_{N/2} & \mathbf{I}_{N/2} & -\mathbf{H}_1 & \mathbf{H}_0 \\ \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & q\mathbf{I}_{N/2} & \mathbf{0}_{N/2} \\ \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & \mathbf{0}_{N/2} & q\mathbf{I}_{N/2} \end{pmatrix}. \quad (13)$$

The upper right part of the matrix $\mathcal{B}_{CS}^{NTRU+}$ does not preserve a symmetry that enables defining a similar homomorphism over integers. One possible homomorphism that halves the lattice dimension is given in Figure 8. However, since the matrices $\mathcal{B}_{CS,+}^{NTRU+}$ and $\mathcal{B}_{CS,-}^{NTRU+}$ are defined over complex numbers, the attacker needs to remap the basis to one defined over integer entries and thus does not gain any advantage from the defined homomorphisms. This interpretation supports the argument in Section 2 for selecting the modulus $x^N + 1$ when N is a power of 2.

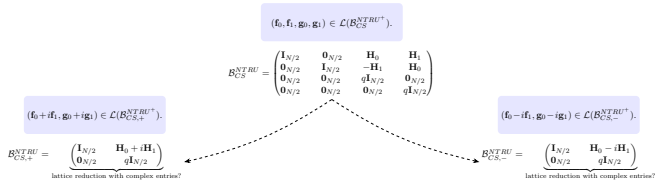


Figure 8: A dimension reduction homomorphism reduces the dimension of the lattice by a factor of 2 but is defined over complex entries where $i = \sqrt{-1}$. This applies when N is a power of 2, and the modulus $x^N + 1$ is used to construct the NTRU ring.

4. Our Construction

Before introducing our construction, we recall the DiTRU [31] construction on the group ring of the dihe-

dral group.

Definition 13 (DiTRU). For a dihedral group of order $2N$ defined as $D_N = \langle x, y \mid x^N = y^2 = 1, xy = yx^{N-1} \rangle$, DiTRU operates over the group ring of the dihedral group

$$\mathcal{R}^{DiTRU} = \mathbb{Z}D_N \approx \frac{\mathbb{Z}[x, y]}{(x^N - 1, y^2 - 1, xy - yx^{N-1})}. \quad (14)$$

And, $\mathcal{R}_q^{DiTRU}$ and $\mathcal{R}_p^{DiTRU}$ represent the rings of DiTRU when the coefficients originate from $\mathbb{Z}_q$ and $\mathbb{Z}_p$ rather than $\mathbb{Z}$.

Multiplication over $\mathcal{R}^{DiTRU}$. A polynomial $f(x) \in \mathbb{Z}C_N$ is represented by a vector of length N as $\mathbf{f} = (f_0, f_1, \dots, f_{N-1})$. Similarly, an element $f = f_0(x) + yf_1(x) \in \mathbb{Z}D_N$ can be represented by a vector $\mathbf{f} = (\mathbf{f}_0, \mathbf{f}_1) \in \mathbb{Z}^{2N}$. The polynomial $\overline{f(x)}$ denotes the conjugate of $f(x)$ that is defined as $\overline{f(x)} = f(x^{N-1})$ and represented as a vector $\overline{\mathbf{f}} = (f_0, f_{N-1}, \dots, f_1)$. For two elements $f = f_0(x) + yf_1(x), g = g_0(x) + yg_1(x) \in \mathbb{Z}D_N$, the multiplication is defined as

$$\begin{aligned} f * g = & f_0(x) * g_0(x) + \overline{f_1(x)} * g_1(x) \\ & + y(f_1(x) * g_0(x) + \overline{f_0(x)} * g_1(x)). \end{aligned} \quad (15)$$

DiTRU lattice. From equation 11, one can construct the lattice of DiTRU generated from the basis

$$\mathcal{B}_{CS}^{DiTRU} = \begin{pmatrix} \mathbf{I}_N & \mathbf{0}_N & \mathbf{H}_0 & \mathbf{H}_1 \\ \mathbf{0}_N & \mathbf{I}_N & \mathbf{H}_1 & \mathbf{H}_0 \\ \mathbf{0}_N & \mathbf{0}_N & q\mathbf{I}_N & \mathbf{0}_N \\ \mathbf{0}_N & \mathbf{0}_N & \mathbf{0}_N & q\mathbf{I}_N \end{pmatrix} \quad (16)$$

where $\mathbf{H}_0$ is a right circulant matrix and $\mathbf{H}_1$ is a left circulant matrix. A homomorphism similar to the one mentioned in Figure 7 can be applied against the lattice. Therefore, one needs almost double the dimension of the lattice to achieve similar security levels compared to NTRU with prime $N' \approx 2N$. The authors of DiTRU mentioned the relevant attack. As a result, the parameters of DiTRU resulted in a construction that is approximately two times slower than NTRU over the ring of integers.

In the following subsection, we introduce our first construction. We call it DiTRU⁺. We look at DiTRU⁺ as the analog of NTRU⁺ over a noncommutative structure.

4.1. KEM from a PPKE over DiTRU⁺

The first construction is simply an adaption of DiTRU by twisting the multiplication over the DiTRU ring according to the following 2-cocycle $\chi : D_N \times D_N \rightarrow \mathbb{Z}_q^*$ defined as

$$\chi(y^{j_1}x^{i_1}, y^{j_2}x^{i_2}) = \begin{cases} -1, & \text{for } i_1, i_2 \in \{0, 1, \dots, n-1\} \\ & \text{and } j_1 = j_2 = 1 \\ 1, & \text{otherwise.} \end{cases} \quad (17)$$

Hence, the ring of DiTRU⁺ is defined as

$$\mathcal{R}^{DiTRU^+} = \mathbb{Z}^x D_N \approx \frac{\mathbb{Z}[x, y]}{(x^N - 1, y^2 + 1, xy - yx^{N-1})}. \quad (18)$$

Similarly, we denote the ring as $\mathcal{R}_q^{DiTRU^+}$ and $\mathcal{R}_p^{DiTRU^+}$ when the coefficients come from $\mathbb{Z}_q$ and $\mathbb{Z}_p$, respectively.

Multiplication over $\mathcal{R}^{DiTRU^+}$. As the multiplication is simply twisted by the rule given in 17, the multiplication over DiTRU⁺ is defined as

$$f * g = f_0(x) * g_0(x) - \overline{f_1(x)} * g_1(x) + y(f_1(x) * g_0(x) + \overline{f_0(x)} * g_1(x)). \quad (19)$$

DiTRU⁺ lattice. From 10, the lattice of DiTRU⁺ is generated from the basis

$$\mathcal{B}_{CS}^{DiTRU^+} = \begin{pmatrix} \mathbf{I}_N & \mathbf{0}_N & \mathbf{H}_0 & \mathbf{H}_1 \\ \mathbf{0}_N & \mathbf{I}_N & -\mathbf{H}_1 & \mathbf{H}_0 \\ \mathbf{0}_N & \mathbf{0}_N & q\mathbf{I}_N & \mathbf{0}_N \\ \mathbf{0}_N & \mathbf{0}_N & \mathbf{0}_N & q\mathbf{I}_N \end{pmatrix}. \quad (20)$$

From a broader perspective, the pattern is similar to NTRU⁺, which does not allow for the definition of homomorphisms to lower dimensions. The key difference lies in the structure of the associated matrices: in NTRU⁺, the public key h is represented by a right circulant matrix with twisted multiplication. In contrast, for DiTRU⁺, the matrix corresponding to the key consists of $\mathbf{H}_0$ as a right circulant matrix and $\mathbf{H}_1$ as a left circulant matrix. Overall, a potential homomorphism that might reduce the dimension is similar to the one illustrated in Figure 8, which is defined using complex numbers. To utilize lattice reduction algorithms like BKZ, it is necessary to expand the dimension and define it with integer entries. Consequently, the defined homomorphism does not effectively assist in reducing the dimension of the lattice in practice. Since DiTRU⁺ is a straightforward adaptation of the DiTRU ring, the inversion algorithm introduced in DiTRU continues to work effectively with minor modifications related to twisting the multiplication according to the rules of the 2-cocycle defined in 17. Additionally, the decryption failure model introduced for DiTRU can also be applied to DiTRU⁺. For completeness, we mention the inversion algorithm for DiTRU⁺. For an input $f = f_0(x) + yf_1(x) \in RD_N$, Algorithm 1 outputs the inverse $f^{-1} = \tilde{f}_0(x) + y\tilde{f}_1(x)$. The input element f is invertible if and only if its determinant, defined as $\det(f) = f_0(x) * \overline{f_0(x)} + f_1(x) * \overline{f_1(x)}$, is a unit over RC_N (that is, the group ring over the cyclic group). For a detailed discussion, refer to [31, Theorem 2].

4.1.1. Parameter selection.

We consider the following attacks to parameterize our cryptosystem.

Search attack. For the GR-NTRU over dihedral group ring, the combinatorial search attack involves finding a

Algorithm 1 Inversion for DiTRU⁺

```

1: Input:  $f = f_0(x) + yf_1(x) \in RD_N$ 
2: Output:  $f^{-1} = \tilde{f}_0(x) + y\tilde{f}_1(x) \in RD_N$  or a failure
3:  $t_1(x) \leftarrow f_0(x) * \overline{f_0(x)}$ 
4:  $t_2(x) \leftarrow f_1(x) * \overline{f_1(x)}$ 
5:  $\det(f) \leftarrow t_1(x) + t_2(x) \triangleright$  Coefficientwise addition in  $R$ 
6:  $t_3(x), found \leftarrow \text{inverseRC}_N(\det(f))$ 
7: if not found then
8:   return failure
9: end if
10:  $\tilde{f}_0(x) \leftarrow \overline{f_0(x)} * t_3(x)$ 
11:  $\tilde{f}_1(x) \leftarrow -\overline{f_1(x)} * t_3(x)$ 
12: return  $f^{-1} = \tilde{f}_0(x) + y\tilde{f}_1(x)$ 

```

small ternary vector (not necessarily exact key $\mathbf{f}$) $\mathbf{f}' = (\mathbf{f}'_0, \mathbf{f}'_1) \in \mathbb{Z}^{2N}$ such that $\mathbf{f}' * \mathbf{H}$ is also small. Therefore, the brute-force search cost equals the size of the space $\mathcal{L}_f$. However, since for any such $\mathbf{f}'$, the $2N$ rotation vectors $(x^i * \mathbf{f}'_0, x^{-i} * \mathbf{f}'_1) * \mathbf{H}$ and $(x^i * \mathbf{f}'_1, x^{-i} * \mathbf{f}'_0) * \mathbf{H}$, for $i = 0, 1, \dots, N-1$, [47, Theorem 5] are also small, thus reducing the search space size by a factor of $2N$. In addition, the meet-in-the-middle techniques [48] further reduce the search cost to

$$\sqrt{\frac{|\mathcal{L}_f|}{2N}} = \sqrt{\frac{1}{2N} \binom{2N}{d} \binom{2N-d}{d+1}}. \quad (21)$$

Primal attack. The most prominent technique to attack lattice-based schemes is to recover the secret information embedded in a lattice by solving hard lattice problems using specially designed lattice reduction algorithms. Therefore, the cost of attacking a lattice-based scheme is measured in terms of the complexity of the best lattice reduction algorithms. The basis created using the public key and other public parameters results in a bad basis, consisting of large and highly non-orthogonal vectors. Lattice reduction algorithms operate on this bad basis with the aim of generating a good one that features shorter and more orthogonal vectors. A good basis functions as a trapdoor, containing private information that aids in solving specific difficult lattice problems.

Some of these algorithms run in polynomial time, such as LLL [49]. However, they are effective in lower dimensions, and the quality of their output degrades with the dimension. For an n -dimensional lattice $\mathcal{L}(\mathbf{B})$, LLL returns a vector v such that $\|v\| \leq 2^{\frac{n-1}{4}} \lambda_1(\mathcal{L}(\mathbf{B}))$. BKZ [50] and its advancements [50, 51] improve upon LLL, produce a better quality reduced basis, and work in higher dimensions; however, their complexity is exponential in the input parameter called blocksize, denoted as β . BKZ with blocksize β is called BKZ- β , and its output is called BKZ- β reduced basis. Formally,

Definition 14 (BKZ). A basis $\mathbf{B} = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1}\}$ is called BKZ- β reduced if

$$\|\mathbf{b}_\kappa^*\| = \lambda_1(\mathcal{L}(\mathbf{B}[\kappa : \min(\kappa + \beta, n)])) \quad (22)$$

for all $\kappa = 0, \dots, n-1$, where $\mathcal{L}(\mathbf{B}[l:r])$, for $0 \leq l < r \leq n$, is the projected sublattice generated by $\{\pi_l(\mathbf{b}_l), \dots, \pi_r(\mathbf{b}_{r-1})\}$.

Clearly, the larger the value of $\beta \leq n$, the shorter the vector $\mathbf{b}_0$ of the BKZ- β reduced basis $\mathbf{B}$. BKZ finds the shortest vector in each of the β -dimensional projected sublattices $\mathcal{L}(\mathbf{B}[\kappa : \min(\kappa + \beta, n)])$ using an SVP oracle. One round of solving SVP for each $\kappa = 0, 1, \dots, n-1$ is called a BKZ tour. BKZ tours are iteratively applied till the condition 22 is satisfied for all the indices. It is known that the number of tours required to obtain a BKZ- β basis is polynomially bounded. Therefore, the cost of BKZ- β is dominated by the cost of calling the SVP oracle. The core-SVP methodology [52] is a pessimistic estimation that assumes only one call to the oracle is enough to solve SVP. Enumeration [53] and Sieving [54] are two well-known techniques employed in the BKZ SVP oracle. It has been observed that sieving outperforms enumeration for $\beta \geq 65$ [55]. Therefore, the lattice security of recent proposals is assessed based on the core-SVP hardness of BKZ with sieving, given by

$$2^{0.292\beta + o(\beta)} (\text{classical}) [54] \text{ and } 2^{0.265\beta + o(\beta)} (\text{quantum}) [56].$$

Since the overall cost of attack depends on the blocksize β , it is important to estimate the value of β for higher dimensions. Various estimators [52, 57, 58] have been introduced in the literature for this purpose. The 2016-estimator [52] is a well-known one that uses the following observation to estimate β : BKZ- β detects a unique short vector $\mathbf{u}$ if

$$\|\pi_{n-\beta}(\mathbf{u})\| \leq \|\mathbf{b}_{n-\beta}^*\|. \quad (23)$$

The left-hand side norm is expected to be $\sqrt{\beta/n} \|\mathbf{u}\|$. For the right-hand side, we depend on assumptions to simulate the profile of the reduced basis. Generally, the profile $\{\mathbf{b}_0^*, \mathbf{b}_1^*, \dots, \mathbf{b}_{n-1}^*\}$ of a BKZ- β reduced basis $\mathbf{B} = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1}\}$ follows the Geometric Series Assumption (GSA) [59] as $\|\mathbf{b}_i^*\| \approx \delta_\beta^{-2} \|\mathbf{b}_{i-1}^*\|$, where the quantity

$$\delta_\beta = \left(\|\mathbf{b}_0\| / \det(\mathbf{B})^{1/n} \right)^{1/n}, \quad (24)$$

is called the root Hermite factor of an n -dimensional BKZ- β reduced basis, that can be estimated by $\delta_\beta = \left(\frac{\beta}{2\pi e} (\pi\beta)^{\frac{1}{\beta}} \right)^{\frac{1}{2(\beta-1)}}$, for $\beta > 50$ [60]. Following GSA, the condition for 2016-estimator can be expressed as:

$$\sqrt{\beta/n} \|\mathbf{u}\| \leq \delta_\beta^{2\beta-n-1} \cdot \det(\mathbf{B})^{1/n}. \quad (25)$$

Ducas et al. [61] gave a more accurate estimator called Fatigue-estimator, utilizing an alternative heuristic Z-GSA for simulating the BKZ- β reduced profile of NTRU-like lattices.

Definition 15 (Z-GSA). [61, Heuristic 2.8] For a $2n$ -dimensional q -ary lattice $\mathcal{L}(\mathbf{B})$ with n q -vectors, the Z-GSA assumption predicts the profile of the lattice as follows:

$$\|\mathbf{b}_i^*\| = \begin{cases} q, & \text{for } i \in [1, n-m], \\ \sqrt{q} \cdot \delta_\beta^{2n-1-2i}, & \text{for } i \in (n-m, n+m), \\ 1, & \text{for } i \in [n+m, 2n], \end{cases} \quad (26)$$

for

$$m = \frac{1}{2} + \frac{\ln(q)}{\ln(\delta_\beta)}.$$

However, both the assumptions GSA and Z-GSA result in the same equation 25 in the cases considered in this work.

Hybrid attack. The hybrid attack on NTRU-like lattices with a ternary secret is well-explained in [62]. Therefore, we refrain from going deep into the technical details; however, we discuss the basic idea behind the attack. The hybrid attack leverages both the lattice reduction techniques and the search attack. The public basis is divided into blocks as follows:

$$\mathcal{B}_{CS} = \begin{pmatrix} q\mathbf{I}_N & \mathbf{0}_N \\ \mathbf{H} & \mathbf{I}_N \end{pmatrix} = \begin{pmatrix} q\mathbf{I}_{r_1} & \mathbf{0} & \mathbf{0} \\ * & \mathbf{H}' & \mathbf{0} \\ * & * & \mathbf{I}_K \end{pmatrix} \quad (27)$$

where $\mathbf{H}'$ is a $r_2 - r_1$ matrix and $K = 2N - r_2$. The attacker first reduces and then lower triangularizes the block $\mathbf{H}'$, which is equivalent to multiplying by an unimodular matrix $\mathbf{M}'$ and an orthogonal matrix $\mathbf{L}'$ to get $\mathbf{M}' * \mathbf{H}' * \mathbf{L}'$. Let $\mathcal{T} = \mathbf{M} * \mathcal{B}_{CS} * \mathbf{L}$ where $\mathbf{M} = \text{diag}(\mathbf{I}_{r_1} \ \mathbf{M}' \ \mathbf{I}_K)$ and $\mathbf{L} = \text{diag}(\mathbf{I}_{r_1} \ \mathbf{L}' \ \mathbf{I}_K)$ are block diagonal matrices. Observe that $(\mathbf{g}, \mathbf{f}) * \mathbf{L}$ lies in the lattice defined by $\mathcal{T}$. Suppose that $\mathbf{s}$ and $\mathbf{v}$ are the first r_2 and last K coordinates of $(\mathbf{g}, \mathbf{f}) * \mathbf{L}$ then there exists an $\mathbf{u} \in \mathbb{Z}^{r_2}$ such that $(\mathbf{u}|\mathbf{v}) * \mathcal{T} = (\mathbf{g}, \mathbf{f}) * \mathbf{L} = (\mathbf{s}|\mathbf{v})$. It can be rewritten as

$$(\mathbf{0}|\mathbf{v}) * \mathcal{T} - (\mathbf{0}|\mathbf{v}) = -(\mathbf{u}|\mathbf{0}) * \mathcal{T} + (\mathbf{s}|\mathbf{0}). \quad (28)$$

Therefore, the attacker guesses the last K coordinates of $\mathbf{f}$ and for the correct guess, i.e., $\mathbf{v}$, if the block $\mathbf{H}'$ is reduced to an extent that $|\mathcal{T}_{i,i}|/2 \geq \|(\mathbf{s}|\mathbf{v})\|_\infty = \|(\mathbf{g}, \mathbf{f}) * \mathbf{L}\|_\infty = 1$ (since $\mathbf{L}$ is orthogonal), then the full private key can be recovered as per [62, Lemma 1]. Further, it is shown in [62] that the meet-in-the-middle technique can be applied to search for $\mathbf{v}$, thus further reducing the search cost. For more details refer to [62, 63].

Overstretched regime. Selecting the very large modulus q leads to improved lattice reduction attacks against the NTRU lattice as independently demonstrated in [64] and [65]. Such a class of NTRU schemes is termed "overstretched". Kirchner and Fouque [66] gave a geometrical interpretation of this phenomenon, called Dense Sublattice Discovery (DSD), arising from discovering a basis for a dense sublattice formed by the rotations of the private key before solving the SVP for a very large q . They proposed $n^{2.783+o(1)}$ as an upper bound for the overstretched regime. Ducas and Woerden [61]

further analyzed the overstretched NTRU lattices, gave the reason for the occurrence of DSD before secret vector recovery, and narrowed the bound to $0.004 \cdot n^{2.484}$ called the Fatigue point. The GR-NTRU lattices over the group ring RG are a generalized form of NTRU lattices containing the sublattice formed by the "rotations" of the private key with respect to the underlying structure. Therefore, overstretched attacks are applicable to GR-NTRU with a modulus q as large as the Fatigue point.

Considering the aforementioned attacks, we summarize the parameters for our construction involving the twisted dihedral group ring. Table 4 presents three sets of parameters targeting 128-bit, 192-bit, and 256-bit security levels³.

4.1.2. KEM construction over DiTRU⁺.

Having clarified the definitions necessary to describe the ring of DiTRU⁺, we now introduce the first PPKE we have considered and the KEM constructed over it. The PPKE is similar to the one introduced in NTRU first-round submission to NIST [10] and DiTRU [31]; we sketch the key generation, encryption, and decryption as in the Figure 9.

KeyGen(1^λ)	Encrypt($h, m, seed_r$)	Decrypt(f, c)
1: initialize Sampler with $\mathcal{L}_{3F+1}$ and $seed_f$	1: initialize Sampler with $\mathcal{L}_r$ and $seed_r$	1: $m' = c * f \pmod{q}$
2: do $f \leftarrow \text{Sampler}$ until f is invertible modulo q	2: $r \leftarrow \text{Sampler}$	2: $t = c - m' \pmod{q}$
3: Let f_q be the inverse of f modulo q	3: $t = pr * h \pmod{q}$	3: initialize Sampler with $\mathcal{L}_m$ and $Hash_1(t)$
4: initialize Sampler with $\mathcal{L}_g$ and $seed_g$	4: initialize Sampler with $\mathcal{L}_m$ and $Hash_1(t)$	4: $m_{mask} \leftarrow \text{Sampler}$
5: $g \leftarrow \text{Sampler}$	5: $m_{mask} \leftarrow \text{Sampler}$	5: $m = m' + m_{mask} \pmod{q}$
6: $h = g * f_q \pmod{q}$	6: $m' = m - m_{mask} \pmod{q}$	6: return m
7: return (f, h)	7: $c = t + m' \pmod{q}$	
	8: return c	

Figure 9: A sketched description of the PPKE for DiTRU⁺.

The value of $d \leq \lfloor 2N/3 \rfloor$ and the corresponding spaces are $\mathcal{L}_g = \mathcal{L}_r = \mathcal{T}(d, d)$, $\mathcal{L}_f = \mathcal{T}(d+1, d)$ and $\mathcal{L}_m = \mathcal{T}_{2N}$. Meanwhile $\mathcal{L}_{3F+1}$ indicates sampling elements of the form $3F+1$ where $F \in \mathcal{L}_f$. Eventually, by applying the FO transformation, we construct the first KEM for DiTRU⁺.

Security reductions. The first construction of the KEM is derived by applying the SVES-3 method [67] to the PPKE sketched in 5. The OW-CPA security of the PPKE relies on the hardness of the GR-NTRU assumption over the DiTRU⁺ ring, and consequently, the IND-CCA2 proof

Encap(h)	Decap($f, c; h$)
1: initialize Sampler with $\mathcal{T}_{2N}$ and $seed_m$	1: $m = \text{Decrypt}(f, c)$
2: $m \leftarrow \text{Sampler}$	2: $seed_r = \text{Hash}_2(h, m)$
3: $seed_r = \text{Hash}_2(h, m)$	3: if $\text{Encrypt}(h, m, seed_r) \neq c$ then
4: $c = \text{Encrypt}(h, m, seed_r)$	4: return $\perp$
5: $k = \text{Hash}_3(m)$	5: else
6: return (c, k)	6: $k = \text{Hash}_3(m)$
	7: end if
	8: return k

Figure 10: A sketched description of the KEM from the PPKE over DiTRU⁺.

of the KEM is supported by the SVES-3 design, which involves masking the message during the encryption process and re-encrypting during the decapsulation process. Masking the message in the encryption routine has two contributions:

- (i) Ensuring that retrieving the message using a lattice reduction algorithm remains difficult, even when the message being encrypted has a low Hamming weight.
- (ii) Supporting the IND-CPA of the underlying encryption scheme. Without masking the message, it becomes evident that when $g = g_0 + yg_1$ belongs to $\mathcal{L}_g = \mathcal{T}(d, d)$, many samples will likely have g_0 and g_1 in $\mathcal{T}(d/2, d/2)$. Consequently, for many instances, we encounter $h(1) \equiv 0 \pmod{q}$, which leads to $c(1) \equiv m(1) \pmod{q}$. This impacts the indistinguishability under chosen plaintext attacks.

Additionally, re-encrypting the message in the decryption routine ensures that the message is correctly retrieved before establishing the shared key k .

In [67], the authors present a non-tight reduction in the Random Oracle Model (ROM) that proves the IND-CCA2 security of this KEM design, contingent on the hardness of the OW-CPA construction of the PPKE schemes in the presence of decryption failures. Similarly, in [68], the authors offer a security proof for the IND-CCA2 security of the underlying KEM, assuming the IND-CPA security of the underlying PPKE without taking decryption failures into account. Overall, the design of this KEM is supported by non-tight security proofs when constructed as it was originally proposed in NTRU-first round submission [10]. A simple modification, as illustrated in Figure 10, leads to tighter security proofs in the ROM. This modified version can also be classified under $\text{KEM}_m^\perp$, as discussed in [40]. The following theorem from [40] describes the IND-CCA security of the KEM constructed from the PPKE over DiTRU⁺.

Theorem 2 (DiTRU⁺: IND-CCA Security in the ROM).

Let PKE be a public key encryption scheme with δ -correctness and γ -spreadness. For any adversary $\mathcal{A}_1$ that attacks the IND-CCA KEM from DiTRU⁺, making at most q_h hash queries and q_d decapsulation queries with

³The script used to select the parameters is available on https://github.com/The-Isogeniest/KEMs_twisted_DiTRU/blob/main/Parameter-Selection.ipynb

Table 4: Parameter sets of twisted dihedral group ring targeting security levels I, III, and IV, respectively. These parameters were chosen based on the Core-SVP methodology, utilizing sieving as an SVP oracle, and employing ZGSA to model the profile of the reduced basis. The decryption failure δ is less than $2^{-\lambda}$ where λ indicates the level of security.

security level	(N, d, q)	Classical						Quantum					
		primal attack		hybrid attack			K	primal attack		hybrid attack			K
		β	cost	β	cost	β		β	cost	β	cost	β	
128	(293, 195, 2048)	456	133	166	437	128	456	121	156	453	120		
192	(443, 159, 2048)	723	211	310	676	197	723	192	293	703	186		
192	(421, 159, 2048)	682	199	287	642	187	682	181	271	668	177		
256	(557, 371, 4096)	898	262	330	892	260	898	238	313	920	244		
256	(547, 364, 4096)	879	257	325	868	253	879	233	305	898	238		

a certain advantage, there exists an adversary $\mathcal{A}_2$ against the IND-CPA PKE schemes constructed from the PPKE over DiTRU^+ that approximately runs in the same time as $\mathcal{A}_1$ such that

$$\text{Adv}_{\text{DiTRU}^+.\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}_1) \leq 3\text{Adv}_{\text{DiTRU}^+.\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}_2) + q_h\delta + \frac{2q_h}{|\mathcal{M}|} + q_d2^{-\gamma}.$$

However, it is essential to note that this construction does not have direct support from tight proofs against a quantum adversary for the specified design, and the IND-CCA security of the KEM in QROM is based on the OW-CPA security of the underlying PKE [37][Theorem 6.1].

4.2. From PPKE to DPKE

As discussed, the KEM introduced earlier is based on non-tight security proofs in the ROM and involves masking the message and heavy computations in the decapsulation, as the encryption routine is called again. Additionally, the key generation procedure includes an invertibility check condition (line 2 in 9) since the sampled f does not need to be invertible for every sample, even though it occurs with high probability.

In the third round of NIST submissions, the KEM of NTRU has been developed based on a design introduced in [69]. This KEM is structured around a DPKE rather than a PPKE, which allows for avoiding the invertibility check and re-encryption in the decryption process. To achieve this, we need to identify a space in which the sampled polynomial f is always invertible within the underlying ring. This raises a motivational question:

Question

Can we provide a similar proof and KEM construction over the noncommutative ring associated with the dihedral group?

We now introduce a modification to the ring structure of DiTRU^+ to support these changes.

Let $\phi_N(x) = x^{N-1} + x^{N-2} + \dots + 1$ be the N^{th} cyclotomic polynomial for prime N . We modify the definition of the

DiTRU^+ ring as follows:

$$\mathcal{R}^{\text{DiTRU}^{\phi^+}} = \frac{\mathbb{Z}[x, y]}{(\phi_N(x), y^2 + 1, xy - yx^{N-1})}. \quad (29)$$

Similarly, we denote the ring as $\mathcal{R}_q^{\text{DiTRU}^{\phi^+}}$ and $\mathcal{R}_p^{\text{DiTRU}^{\phi^+}}$ when the coefficients come from $\mathbb{Z}_q$ and $\mathbb{Z}_p$, respectively.

Multiplication over $\mathcal{R}^{\text{DiTRU}^{\phi^+}}$. Before providing the multiplications, we give the following definition.

Definition 16 (Coefficient vectors). Let $r(x) = r_0 + r_1x + \dots + r_{N-1}x^{N-1} \in \mathbb{Z}[x]/(x^N - 1)$ and $s(x) = s_0 + s_1x + \dots + s_{N-2}x^{N-2} \in \mathbb{Z}[x]/(\phi_N(x))$. Then,

1. The coefficient vector of $r(x)$ is defined as $\mathbf{r} = (r_0, r_1, \dots, r_{N-1}) \in \mathbb{Z}^N$. Similarly, the coefficient vector of $s(x)$ is defined as $\mathbf{s} = (s_0, s_1, \dots, s_{N-2}) \in \mathbb{Z}^{N-1}$.
2. The coefficient vector of image of $r(x)$ in $\mathbb{Z}[x]/(\phi_N(x))$, i.e., $r(x) \bmod \phi_N(x) = (r_0 - r_{N-1}) + (r_1 - r_{N-1})x + \dots + (r_{N-2} - r_{N-1})x^{N-2}$ is denoted by $\mathbf{r}_\phi = (r_0 - r_{N-1}, r_1 - r_{N-1}, \dots, r_{N-2} - r_{N-1}) \in \mathbb{Z}^{N-1}$. On the other hand, $s(x)$ can be embedded into $\mathbb{Z}[x]/(x^N - 1)$ as $\text{Lift}(s(x)) = s_0 + s_1x + \dots + s_{N-2}x^{N-2} + 0x^{N-1}$, and its coefficient vector is given by $\text{Lift}(\mathbf{s}) = (\mathbf{s}, 0) = (s_0, s_1, \dots, s_{N-2}, 0) \in \mathbb{Z}^N$.

For any polynomial $f(x)$ of maximum degree $N - 2$, represented in the vector form as $\mathbf{f} = (f_0, f_1, \dots, f_{N-2})$, the conjugate $\overline{f(x)}$ modulo $\phi_N(x)$ is given by $\mathbf{f} = (f_0 - f_1, -f_1, f_{N-2} - f_1, \dots, f_2 - f_1)$.

Furthermore, multiplication is defined similarly as in 19, with the main difference being that the multiplication of polynomials is performed modulo $\phi_N(x)$ instead of $x^N - 1$. In simpler terms, for any two polynomials $f(x)$ and $g(x)$, each with a maximum degree of $N - 2$, the result of their multiplication modulo $\phi_N(x)$ can be obtained by first performing standard polynomial multiplication modulo $x^N - 1$ as $\text{Lift}(f(x)) * \text{Lift}(g(x))$ and then subtracting the leading coefficient from all other coefficients, i.e., getting $(\text{Lift}(f(x)) * \text{Lift}(g(x))) \bmod \phi_N(x)$.

DiTRU^{ϕ^+} lattice. The ring $\mathcal{R}^{\text{DiTRU}^{\phi^+}}$ cannot be directly represented as a group ring. The basis matrix of the

lattice associated with DiTRU^{ϕ^+} follows the same pattern as that of DiTRU^+

$$\mathcal{B}_{CS}^{\text{DiTRU}^{\phi^+}} = \begin{pmatrix} \mathbf{I}_{N-1} & \mathbf{0}_{N-1} & \mathbf{H}_0^\phi & \mathbf{H}_1^\phi \\ \mathbf{0}_{N-1} & \mathbf{I}_{N-1} & -\mathbf{H}_1^\phi & \mathbf{H}_0^\phi \\ \mathbf{0}_{N-1} & \mathbf{0}_{N-1} & q\mathbf{I}_{N-1} & \mathbf{0}_{N-1} \\ \mathbf{0}_{N-1} & \mathbf{0}_{N-1} & \mathbf{0}_{N-1} & q\mathbf{I}_{N-1} \end{pmatrix}. \quad (30)$$

However, in this case, $\mathbf{H}_0^\phi$ and $\mathbf{H}_1^\phi$ are right and left circulant matrices modulo $\phi_N(x)$ instead of modulo $x^N - 1$. Let $h = h_0(x) + y h_1(x)$ be the public key for DiTRU^{ϕ^+} , then the matrices $\mathbf{H}_0^\phi$ and $\mathbf{H}_1^\phi$ are given by

$$\mathbf{H}_0^\phi = \begin{pmatrix} \mathbf{h}_0 \\ (Lift(x) * Lift(h_0(x)))_\phi \\ (Lift(x^2) * Lift(h_0(x)))_\phi \\ \vdots \\ (Lift(x^{N-2}) * Lift(h_0(x)))_\phi \end{pmatrix} \text{ and } \mathbf{H}_1^\phi = \begin{pmatrix} \mathbf{h}_1 \\ (Lift(x^{N-1}) * Lift(h_1(x)))_\phi \\ (Lift(x^{N-2}) * Lift(h_0(x)))_\phi \\ \vdots \\ (Lift(x^2) * Lift(h_0(x)))_\phi \end{pmatrix},$$

that is, after the expansion

$$\mathbf{H}_0^\phi = \begin{pmatrix} h_{00} & h_{01} & \dots & h_{0N-3} & h_{0N-2} \\ -h_{0N-2} & h_{00} - h_{0N-2} & \dots & h_{0N-4} - h_{0N-2} & h_{0N-3} - h_{0N-2} \\ h_{0N-2} - h_{0N-3} & -h_{0N-3} & \dots & h_{0N-5} - h_{0N-3} & h_{0N-4} - h_{0N-3} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ h_{03} - h_{02} & h_{04} - h_{02} & \dots & h_{00} - h_{02} & h_{01} - h_{02} \\ h_{02} - h_{01} & h_{03} - h_{01} & \dots & -h_{01} & h_{00} - h_{01} \end{pmatrix},$$

and

$$\mathbf{H}_1^\phi = \begin{pmatrix} h_{10} & h_{11} & \dots & h_{1N-3} & h_{1N-2} \\ h_{11} - h_{10} & h_{12} - h_{10} & \dots & h_{1N-2} - h_{10} & -h_{10} \\ h_{12} - h_{11} & h_{13} - h_{11} & \dots & -h_{11} & h_{10} - h_{11} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ h_{1N-3} - h_{1N-4} & h_{1N-2} - h_{1N-4} & \dots & h_{1N-5} - h_{1N-4} & h_{1N-3} - h_{1N-4} \\ h_{1N-2} - h_{1N-3} & -h_{1N-3} & \dots & h_{1N-5} - h_{1N-3} & h_{1N-4} - h_{1N-3} \end{pmatrix}.$$

Invertibility check. As we discussed earlier, the key generation process includes an invertibility check that involves sampling an element f from the corresponding space until we find one that is invertible. In the following lemma, we show that the invertibility check condition for any randomly sampled element f from $\mathcal{R}_q^{\text{DiTRU}^{\phi^+}}$, where q is a prime (or a power of prime p), can be brought down to checking if $\det(f)$ is nonzero modulo q (or p).

Lemma 3 (Invertibility). *Let q be a prime (or a power of a prime p), and the order of q (or p) in $\mathbb{Z}_N^*$ is $N - 1$ for an odd prime N . Then any element $f = f_0(x) + y f_1(x) \in \mathcal{R}_q^{\text{DiTRU}^{\phi^+}}$ such that $\det(f) = f_0(x) * \overline{f_0(x)} + f_1(x) * \overline{f_1(x)} \not\equiv 0 \pmod{q}$ (or p) is a unit.*

Proof. Let $f = f_0(x) + y f_1(x) \in \mathcal{R}_q^{\text{DiTRU}^{\phi^+}}$, then f is invertible if and only if $\det(f) = f_0(x) * \overline{f_0(x)} + f_1(x) * \overline{f_1(x)}$ is invertible over $\mathbb{Z}_q[x]/(\phi_N(x))$ (line 6 of Algorithm 1). The rest of the proof follows as outlined below.

(i) **Known fact.** Let q be a prime power, and F_q be a finite field of order q . The cyclotomic polynomial $\phi_N(x)$ is irreducible over F_q if and only if N is equal to $2, 4, r^k$ or $2r^k$, where r is an odd prime, k is a positive integer, and q is a primitive root modulo N [70].

(ii) Consequently, if q is prime and the order of q is $N - 1$ in $\mathbb{Z}_N^*$, then $\mathbb{Z}_q[x]/(\phi_N(x))$ is a field and any non-zero polynomial in $\mathbb{Z}_q[x]/(\phi_N(x))$ is a unit. For the case when $q = p^r$ is a prime power and the order of p is $N - 1$ in $\mathbb{Z}_N^*$, $\mathbb{Z}_p[x]/(\phi_N(x))$ is a field. Therefore, the inverse of $\det(f)$ is first computed in $\mathbb{Z}_p[x]/(\phi_N(x))$. This inverse is then deterministically lifted to $\mathbb{Z}_q[x]/(\phi_N(x))$ [71]. $\square$

In our case, q is a power of 2, where 2 is a primitive root modulo an odd prime N . Therefore, $\phi_N(x)$ is irreducible over $\mathbb{Z}_2$ and thus $\mathbb{Z}_2[x]/(\phi_N(x))$ is a field. From Lemma 3, it follows that for any f sampled from $\mathcal{T}(d+1, d)$ or $\mathcal{T}(d, d)$, f is a unit in $\mathbb{Z}_q[x]/(\phi_N(x))$ if and only if $\det(f) \not\equiv 0 \pmod{2}$.

In our construction of DiTRU^{ϕ^+} , we avoid the usual invertibility check condition that needs to execute the complete inversion algorithm to check the invertibility of an element; instead, we reject any randomly sampled element if its determinant is zero modulo 2. This way, we handle the inversion modulo q with less computational overhead. However, we still need to address the invertibility modulo the other modulus p , which is typically set to 3. We have multiple ways to handle this situation. The first approach is to choose f of the form $3F + 1$, where $F \in \mathcal{T}(d, d)$, that eliminates the requirement for f to be invertible modulo 3. However, this results in larger values of q , leading to increased sizes of the public key and ciphertext. The other way is also to select 3 to be a primitive root modulo N so that $\mathbb{Z}_3[x]/(\phi_N(x))$ is a field, and any nonzero element will be invertible. But this makes the possible values for the parameter N sparse, leading to overestimated security parameters and a slower performance. Or, we can simply put a usual invertibility check condition modulo 3. Although this adds some computational overhead, it can help reduce memory requirements. Further, empirical evidence shows that any randomly sampled $f \in \mathcal{T}(d+1, d)$ is always invertible in $\mathbb{Z}_3[x]/(\phi(x))$. Therefore, we prefer to adopt the latter approach for our scheme.

Finally, Table 5 presents the parameter sets for $\mathcal{R}_q^{\text{DiTRU}^{\phi^+}}$ that achieve security levels I, III, and V. Additionally, Table 6 provides the parameter sets that offer security levels comparable to those introduced by NTRU HPS in NIST PQC Round 3. Both sets are correct, i.e., they do not allow decryption failure.

Table 5: Parameter sets of the DPKE KEM over the twisted dihedral group ring targeting I, III, and V levels of security.

Core-SVP	(N, d, q)	Classical						Quantum					
		primal attack			hybrid attack			primal attack			hybrid attack		
		β	cost	K	β	cost		β	cost	K	β	cost	
128	(317, 127, 2048)	486	142	200	456	133		486	129	189	473	125	
192	(443, 255, 4096)	678	198	255	666	194		678	180	240	689	183	
192	(419, 255, 4096)	654	186	236	625	182		637	169	223	644	171	
256	(587, 255, 4096)	934	273	379	907	265		934	248	347	941	249	
256	(563, 255, 4096)	891	260	357	868	253		891	236	337	899	239	

Table 6: Parameter sets of the DPKE KEM over the twisted dihedral group ring achieving equivalent security levels compared to the parameter sets of NTRU HPS introduced in NIST third round.

Core-SVP	(N, d, q)	Classical						Quantum					
		primal attack			hybrid attack			primal attack			hybrid attack		
		β	cost	K	β	cost		β	cost	K	β	cost	
105	(269, 127, 2048)	401	117	156	380	111		401	106	147	394	104	
144	(347, 127, 2048)	540	158	230	503	147		540	143	217	522	138	
178	(419, 255, 4096)	637	186	236	624	182		637	169	223	644	178	

4.2.1. KEM construction over $DiTRU^{\phi^+}$.

In the following, we introduce the new KEM construction, inspired by NTRU third-round submission, over the new noncommutative structure $\mathcal{R}_q^{DiTRU^{\phi^+}}$. The design of this KEM was initially introduced in NTRU-HRSS [69] and is based on a DPKE rather than a PPKE compared to the first KEM presented in Subsection 4.1. Figures 11 and 12 give the DPKE and the KEM constructed over it, respectively where $\mathcal{L}_f = \mathcal{T}(d+1, d)$, $\mathcal{L}_g = \mathcal{L}_m = \mathcal{T}(d, d)$ and $\mathcal{L}_r = \mathcal{T}_{2N}$.

Security reductions. The construction of the KEM closely follows the approach introduced in the third round of NTRU, with necessary modifications to support the noncommutative ring of $DiTRU^{\phi^+}$. This KEM framework is designed to be IND-CCA2 secure, supported by a *tight* reduction in the ROM and a non-tight reduction QROM. The latter can potentially be tightened under additional assumptions, as discussed in [72]. The KEM is built using a DPKE scheme instead of a traditional PPKE scheme. This design choice incorporates implicit rejection and avoids re-encryption in the decapsulation process. By implicit rejection, we mean that the KEM returns a pseudorandom key in the event of decryption failure, rather than the error symbol $\perp$. Moreover, in [73], the authors demonstrate that implicit rejection can be utilized to transform a rigid DPKE into an IND-CCA2 KEM. A DPKE is referred to as rigid if it satisfies the following properties for all keys (sk, pk) , messages m , and ciphertexts c :

- (i) $[\text{Encrypt}(pk, m) = c] \Rightarrow [\text{Decrypt}(sk, c) = m]$; this condition can be easily verified for correct parameter sets.
- (ii) $[\text{Decrypt}(sk, c) = m] \Rightarrow [\text{Encrypt}(pk, m) = c]$; this

can be verified by re-encrypting during the decapsulation process, as introduced in Figure 9, or by checking that $(r, m) \in \mathcal{L}_r \times \mathcal{L}_m$, as shown in the decryption routine in Figure 11 (line 4).

Alternatively, our KEM from the DPKE over $DiTRU^{\phi^+}$ can be shown as an application of $U_m^{\mathcal{X}}$ as introduced in [40], and therefore the following theorem describes the security proof in the ROM.

Theorem 4 ($DiTRU^{\phi^+}$: IND-CCA Security in the ROM). *Let PKE be a deterministic and rigid public key encryption scheme with perfect correctness (i.e., $\delta = 0$). For any adversary $\mathcal{A}_1$ that attacks the IND-CCA KEM from $DiTRU^{\phi^+}$, making at most q_d decapsulation queries with a certain advantage, there exists an adversary $\mathcal{A}_2$ against the OW-CPA PKE schemes constructed from the DPKE over $DiTRU^{\phi^+}$ that approximately runs in the same time as $\mathcal{A}_1$ such that*

$$\text{Adv}_{DiTRU^{\phi^+}.KEM}^{IND-CCA}(\mathcal{A}_1) \leq \text{Adv}_{DiTRU^{\phi^+}.PKE}^{OW-CPA}(\mathcal{A}_2) + \frac{q_d}{|\mathcal{M}|}.$$

4.3. KEM over $DiTRU^+$ and the Eisenstein integers.

Let $\omega = e^{2\pi i/3}$ be the primitive cube root of unity. The ring of Eisenstein integers is defined as the set of complex numbers of the form

$$\mathbb{Z}[\omega] = \{a + b\omega : a, b \in \mathbb{Z}\}.$$

The usual multiplication of two Eisenstein integers, $(a + b\omega) * (c + d\omega)$, is given by $(ac - bd) + (bc + ad - bd)\omega$ and requires four integer multiplications. However, Jarvis et al. [74] demonstrated that by rearranging the terms, we can express the multiplication as $(a + b\omega) * (c + d\omega) = (ac - bd) + (ac + (a - b)(c - d))\omega$, which only requires three integer multiplications.

<u>KeyGen₁(1^λ)</u>	<u>Encrypt(h, m, seed_r)</u>	<u>Decrypt((f, f_p, h_q), c)</u>
1: initialize Sampler with $\mathcal{L}_f$ and $seed_f$ until f is invertible modulo p and q .	1: initialize Sampler with $\mathcal{L}_r$ and $seed_r$	1: $a = c * f$ in $\mathcal{R}_q^{DiTRU^+}$
2: Let f_q be the inverse of f in $\mathcal{R}_q^{DiTRU^+}$	2: $r \leftarrow$ Sampler	2: $m = a * f_p$ in $\mathcal{R}_p^{DiTRU^+}$
3: initialize Sampler with $\mathcal{L}_g$ and $seed_g$	3: $c = r * h + m$ in $\mathcal{R}_q^{DiTRU^+}$	3: $r = ((c - m) * h_q)$ in $\mathcal{R}_q^{DiTRU^+}$
4: $h = 3g * f_q$ in $\mathcal{R}_q^{DiTRU^+}$	4: return c	4: if $r \in \mathcal{L}_r$ and $m \in \mathcal{L}_m$ then
5: $h_q = (1/h)$ in $\mathcal{R}_q^{DiTRU^+}$		5: return $(r, m, success)$
6: $f_p = (1/f)$ in $\mathcal{R}_p^{DiTRU^+}$		6: else
7: return $((f, f_p, h_q), h)$		7: return $(0, 0, failure)$
		8: end if

Figure 11: A sketched description of the DPKE for DiTRU⁺.

<u>KeyGen(1^λ)</u>	<u>Encap(h)</u>	<u>Decap((f, f_p, h_q, s), c)</u>
1: $((f, f_p, h_q), h) \leftarrow$ KeyGen ₁ (1 ^λ)	1: initialize Sampler with $\mathcal{L}_m$ and $seed_m$	1: $(r, m, out) =$ Decrypt $((f, f_p, h), c)$
2: $s \leftarrow^{\$} \{0, 1\}^{256}$	2: $m \leftarrow^{\$}$ Sampler	2: $k_1 = Hash_1(r, m)$
3: return $((f, f_p, h_q, s), h)$	3: $c = \text{Encrypt}(h, m, seed_r)$	3: $k_2 = Hash_2(s, c)$
	4: $k = Hash_1(r, m)$	4: if $out == success$ then
	5: return (c, k)	5: return k_1
		6: else
		7: return k_2
		8: end if

Figure 12: A sketched description of the KEM from the DPKE over DiTRU⁺.

As a result, the convolutional multiplication of two vectors $f, g \in \mathbb{Z}[\omega]^n$ (which are integral vectors of length $2n$) can be performed using $3n^2$ integer multiplications. In contrast, the multiplication of two integral vectors of equal length, $f', g' \in \mathbb{Z}^{2n}$, requires $4n^2$ integer multiplications. Therefore, replacing the traditional ring of integers with the ring of Eisenstein integers in the GR-NTRU framework achieves a performance enhancement of approximately 4/3 times.

Theorem 5. [74, Theorem 1] *All units of $\mathbb{Z}[\omega]$ are exactly included in the set $\{\pm 1, \pm \omega, \pm \omega^2\}$. Additionally, any prime in $\mathbb{Z}[\omega]$ can be generated, up to multiplication by a unit, from the following: the element $1 - \omega$, primes $p \in \mathbb{Z}$ such that $p \equiv 2 \pmod{3}$, and primes $q \in \mathbb{Z}[\omega]$ for which $|q|^2 = p$ where $p \equiv 1 \pmod{3}$.*

Note. For a prime $q \in \mathbb{Z}[\omega]$, the ring $\mathbb{Z}[\omega]/\langle q \rangle$ is a finite field with $|q|^2$ elements [74]. Therefore, by Theorem 5, for an odd prime N , $\phi_N(x)$ is irreducible over $\mathbb{Z}[\omega]/\langle q \rangle$, when $q \in \mathbb{Z}[\omega]$ is a prime such that $|q|^2 = p$ is a rational prime and $p \equiv 1 \pmod{3}$, if and only if p is a primitive root modulo N , since $\mathbb{Z}[\omega]/\langle q \rangle \cong \mathbb{Z}_p$. On the other hand, for a rational Eisenstein prime q , i.e., q is prime such that $q \equiv 2 \pmod{3}$, $\phi_N(x)$ is irreducible over $\mathbb{Z}[\omega]/\langle q \rangle$ if and only if q^2 is a primitive root modulo N , or in other words, order of q^2 modulo N is $N - 1$. However, for any prime q , $(q^2)^{\frac{N-1}{2}} = q^{N-1} = 1 \pmod{N}$ (Euler's theorem). Therefore, for any odd prime N and a rational prime Eisenstein q , $\phi_N(x)$ is not irreducible over the field $\mathbb{Z}[\omega]/\langle q \rangle$.

Consequently, we can design a DPKE with twisted dihedral over Eisenstein integers similar to DiTRU⁺ as in

Figure 11 using the prime modulus q such that $|q|^2 = p$ is a rational prime and $p \equiv 1 \pmod{3}$. However, only a PPKE can be constructed similar to DiTRU⁺ as in Figure 9 using rational Eisenstein prime modulus.

It is worth mentioning that division by Eisenstein integers is a nontrivial operation. A general method to divide by Eisenstein integers is provided in [74, Algorithm 1], but it is expensive, requiring 27 integer multiplications. A more efficient division by rational Eisenstein integers is given in [33] that needs only 4 integer multiplications. Definitely, for a prime $q = q_0 + q_1\omega$ of the form $|q|^2 = p$ where $p \equiv 1 \pmod{3}$ is a prime, $q_1 \neq 0$, thus division by such elements would be costly. Therefore, it is a design choice between DPKE, which has expensive modular operations, and PPKE, which has efficient modular division.

Set-up. For the proof-of-concept, here, we construct only the PPKE for DiTRU⁺ over Eisenstein integers similarly to the Figure 9, followed by KEM as in Figure 10 with the required changes in the underlying rings and the sample spaces. We call this variant to be E-DiTRU⁺. For the above-discussed reasons, we work with the prime rational Eisenstein moduli, and the other parameters and spaces are as follows:

Let $N > 2$ be a prime number and $p, q \in \mathbb{Z}[\omega]$ are rational primes such that $|p| \ll |q|$, and p is fixed to be 2. E-DiTRU⁺ operates over the following rings:

$$\mathcal{R}^{E-DiTRU^+} = \frac{\mathbb{Z}[\omega][x, y]}{(x^N - 1, y^2 + 1, xy - yx^{N-1})}, \quad (31)$$

and $\mathcal{R}_p^{E-DiTRU^+}$ and $\mathcal{R}_q^{E-DiTRU^+}$ when the coefficients belong to $\mathbb{Z}[\omega]/\langle p \rangle$ and $\mathbb{Z}[\omega]/\langle q \rangle$, respectively.

To define the sampling spaces $\mathcal{L}_F, \mathcal{L}_f, \mathcal{L}_g, \mathcal{L}_r \subset \mathcal{R}^{E-DiTRU^+}$, we consider the following set-up:

- $t \leq \lfloor \frac{2}{3}(2N) \rfloor$ and s as the nearest multiple of 3 to t .
- $\mathcal{Z}_n = \{z \in \mathbb{C} \mid z^n = 1\}$, in particular $\mathcal{Z}_3 = \{1, \omega, \omega^2\}$ and $\mathcal{Z}_6 = \{\pm 1, \pm \omega, \pm \omega^2\}$.

The set $\mathcal{L}_F$ consists of elements with exactly t nonzero coefficients from the set $\mathcal{Z}_6$, and $\mathcal{L}_f = \{1 + pF : F \in \mathcal{L}_F\}$. The sets $\mathcal{L}_g, \mathcal{L}_r$ consist of elements with $s/3$ disjoint triples of coefficients, each triple containing elements randomly drawn from either $\mathcal{Z}_3$ or $-\mathcal{Z}_3$. Finally, the message space $\mathcal{L}_m$ is the set of elements with coefficients in $\mathcal{Z}_3 \cup \{0\}$.

$E-DiTRU^+$ lattice. The matrix of the public key $h \in \mathcal{R}^{E-DiTRU^+}$ has the same pattern as the elements of the ring $\mathcal{R}^{DiTRU^+}$, i.e.,

$$\mathbf{H}_\omega = \begin{pmatrix} \mathbf{H}_0 & \mathbf{H}_1 \\ -\mathbf{H}_1 & \mathbf{H}_0 \end{pmatrix} \in M_{2N}(\mathbb{Z}[\omega]). \quad (32)$$

However, to use lattice reduction algorithms, we need to map this matrix to a matrix with real entries. This can be done by observing the following:

An Eisenstein integer $a + b\omega$ can be identified uniquely with its integral vector $(a, b) \in \mathbb{Z}^2$ and its integral matrix $\begin{pmatrix} a & b \\ -b & a-b \end{pmatrix} \in M_2(\mathbb{Z})$. The multiplication of two Eisenstein numbers $(a + b\omega) * (c + d\omega)$ can be realized as $(a, b) * \begin{pmatrix} c & d \\ -d & c-d \end{pmatrix}$. Consequently, the multiplication $f * G$ where $f \in \mathbb{Z}[\omega]^n$ and $G \in M_n(\mathbb{Z}[\omega])$ is equivalent to $\mathbf{f} * \langle G \rangle$ where $\mathbf{f} \in \mathbb{Z}^{2n}$ is obtained by replacing every entry of f with its integral vector and $\langle G \rangle \in M_{2n}(\mathbb{Z})$ is obtained by replacing every entry of G with its integral matrix.

Hence, a lattice attack on the private key is equivalent to solving SVP in an $8N$ -dimensional lattice generated by the matrix

$$\mathcal{B}_{CS}^{E-DiTRU^+} = \begin{pmatrix} \mathbf{I}_{4N} & \langle \mathbf{H}_\omega \rangle \\ \mathbf{0}_{4N} & q\mathbf{I}_{4N} \end{pmatrix} \in M_{8N}(\mathbb{Z}). \quad (33)$$

Inversion. A division by Eisenstein integers of the form $q = q + 0\omega$, called rational Eisenstein, that distributes the remainder around the origin is given in [33]. The same work also proposed a constant-time inversion algorithm [33, Algorithm 3] for ring $\mathbb{Z}[\omega]/\langle q \rangle C_n$, where q is a prime rational Eisenstein ([74, Theorem 1]). The algorithm is a direct adaptation of the constant-time modular inversion over $\mathbb{Z}_q C_n$ (q : prime) by Bernstein and Yang [75]. For $E-DiTRU^+$, we use [33, Algorithm 3] in

the Algorithm 1 to compute the inverses over the ring $\mathcal{R}_q^{E-DiTRU^+}$.

Hybrid Attack on $E-DiTRU^+$. As discussed earlier, the cost of the hybrid attack depends on both the cost of lattice reduction and the search complexity. For standard NTRU with a ternary secret, the search space size is estimated as $2^{H(p)}$, where $p : \mathcal{T}_K \rightarrow \mathbb{R}$ represents the probability mass function induced on $\mathcal{T}_K$ by uniformly at random sampling from the key space $\mathcal{L}_f$. Here, $H(p)$ denotes the Shannon entropy of p .

In the case of $E-DiTRU^+$, the secret key $f \in \mathcal{L}_f \subset \mathbb{Z}[\omega]^{2N}$ has exactly t nonzero coefficients drawn from the set $U_6 = \{\pm 1, \pm \omega, \pm \omega^2\}$. Consequently, the integral vector $\mathbf{f} \in \mathbb{Z}^{4N}$ consists of coefficients of the form $\{(0, 0), (\pm 1, 0), (0, \pm 1), (\mp 1, \mp 1)\}$. Searching for a ternary subsequence of length $2K$ in $\mathbf{f}$ is equivalent to searching for a subsequence of length K in f . To simplify our analysis, rather than estimating the search space size based on entropy, we make the following conservative assumption:

For a uniformly sampled $f \in \mathcal{L}_f$, any K -length subsequence contains approximately $Kt/2N$ nonzero entries from U_6 , with each nonzero element appearing approximately $t_K = \frac{1}{6} \left(\frac{Kt}{2N} \right)$.

Under this assumption, we conservatively estimate the search space size as

$$\sqrt{\frac{1}{2N} \prod_{i=0}^5 \binom{K - it_K}{t_K}}. \quad (34)$$

The lower search cost results in higher values of K and a larger gap between the costs of the hybrid attack and the primal attack compared to NTRU, as reflected in Table 7.

Decryption failure. The decryption failures can be eliminated by selecting q such that $|q| > \frac{32N}{3} + 2$. However, for selected N, t and q , the probability of decryption failure can be modeled as

$$P_{dec_fail}(N, q, t) = 1 - \left(1 - \exp \left(- \frac{|q|^2}{8\sigma^2} \right) \right)^{2N}, \quad (35)$$

where $\sigma^2 = \frac{t(3N+2t)}{2N} + \frac{3}{8}$, following the similar steps as in [74, 33].

5. Performance Benchmark

To evaluate our constructions, we provide implementations for the three proposed KEMs in this work:

1. The PPKE over the twisted dihedral group and the ring of integers.
2. The PPKE over the twisted dihedral group and the ring of Eisenstein integers.
3. The DPKE over the twisted dihedral group and the ring of integers.

Table 7: Parameter sets of E-DiTRU⁺ targeting security levels I, III, and V, respectively. These parameters are chosen based on the Core-SVP methodology, utilizing sieving as an SVP oracle, and employing ZGSA to model the profile of the reduced basis.

security level	(N, t, q)	Classical						Quantum					
		primal attack		hybrid attack				primal attack		hybrid attack			
		β	cost	K	β	reduction cost	search cost	β	cost	K	β	reduction cost	search cost
128	(149, 132, 491)	509	148	272	436	127	131	509	134	262	453	120	121
192	(211, 204, 761)	733	214	376	657	191	196	733	194	356	690	183	183
256	(263, 326, 1151)	926	270	426	872	254	257	926	245	406	907	240	239

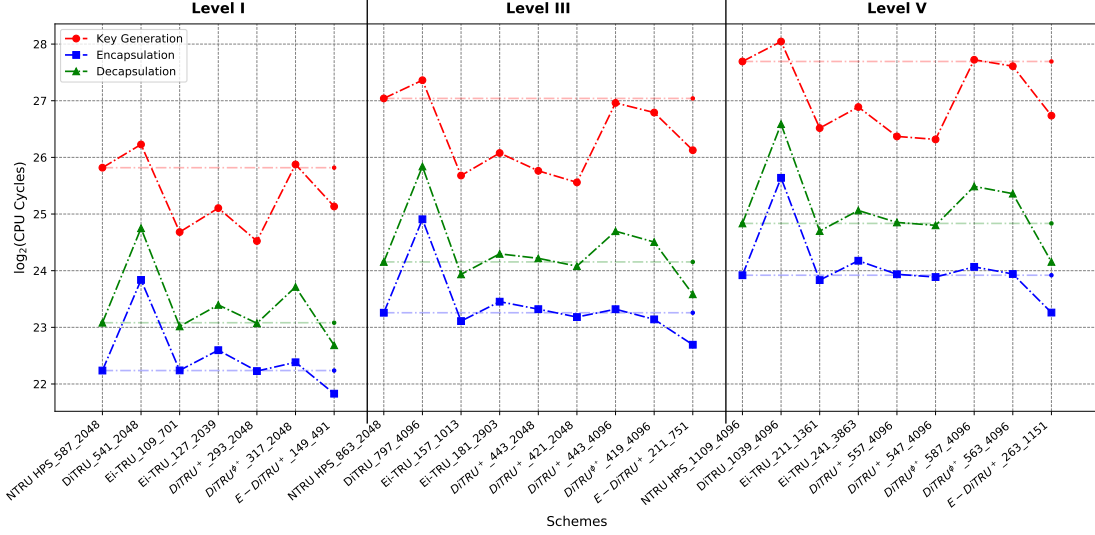


Figure 13: Performance comparison of Key generation, Encapsulation, and Decapsulation of our constructions with other NTRU-like schemes in the literature. NTRU-HPS [9] with parameters revised in [31] is used as a benchmark for comparison.

The implementations of the PPKE KEMs are inspired by the NIST first-round submission of NTRU [10]. In contrast, the DPKE implementation is based on the NIST third-round submission of NTRU-HPS [5]. We ran our implementation on a single-core system with a 4.7 GHZ Intel Core i7-1255U CPU and 32 GB of RAM. The operating system used during the measurements was Ubuntu 22.04.3 LTS. Turbo Boost and multi-threading were disabled as a standard practice while recording the measurements. We utilized GCC 11.4.0 as the compiler with no optimization flag enabled. Tables 2, 3, and Figure 13 present the CPU cycles required for key generation, encapsulation, and decapsulation along with the memory requirements for our constructions proposed in this work, comparing them against the NTRU-HPS [10] as a commutative design of the KEM, along with DiTRU [31] and Ei-TRU [33] as non-commutative instances of the design.

We consider the NTRU-HPS design, derived from a PPKE scheme, as a reference point for comparison. The proposed designs of noncommutative NTRU, namely DiTRU and Ei-TRU, have been built upon the PPKE framework. Additionally, we report the results for the KEM from the DPKE construction. For the KEM constructed from the DPKE, as the construction is based on the third-round of NTRU submitted to NIST [5], we propose two sets of parameters: one achieves the revised se-

Table 8: Comparison of CPU cycles ($\times 10^3$) required for Key generation, Encapsulation, and Decapsulation of the key for NTRU-HPS [5] (commutative), and the construction of the KEM from DPKE proposed in this work (noncommutative). The schemes are grouped into three categories according to their blocksize β and security estimation as per the parameter sets provided in [5].

	KeyGen	Encap	Decap	Ratio
NTRU-HPS_2048_509	87 574	3 651	8 890	(1.00: 1.00: 1.00)
DiTRU ⁺ _269_2048 [This work, Table 6]	49 357	4 062	9 912	(0.56: 1.11: 1.11)
NTRU-HPS_2048_677	1 53 546	6 127	15 708	(1.00: 1.00: 1.00)
DiTRU ⁺ _347_2048 [This work, Table 6]	79 710	6 614	16 759	(0.52: 1.08: 1.07)
NTRU-HPS_4096_821	2 13 948	8 807	22 928	(1.00: 1.00: 1.00)
DiTRU ⁺ _419_4096 [This work, Table 6]	1 14 804	9 306	23 989	(0.54: 1.06: 1.05)

curity levels as defined by NIST, and the other provides security levels similar to those provided in the third-round of NTRU. The measurements for the revised parameter sets have been reported in Table 3 while the comparison for the second parameter set is listed in Table 8.

5.1. Analysis and Discussion

From the results provided, we can observe that our constructions offer efficient and compact candidates in non-commutative algebra, outperforming both the commutative and noncommutative candidates of NTRU.

- **Key Generation:** Our three constructions using noncommutative algebra demonstrate a fast key gen-

eration process. The fastest key generation among all measured variants in this paper is recorded for our construction, DiTRU⁺, which is approximately 2.4X faster than NTRU-HPS.

- **Encapsulation:** The encapsulation process for our KEM, derived from the PPKE over Eisenstein, E-DiTRU⁺, is notably efficient. It is observed to be 1.3X, 1.5X, and 1.6X faster than the KEM based on the PPKE from NTRU-HPS at the first, third, and fifth security levels, respectively.

Additionally, the PPKE KEM over DiTRU⁺ and the DPKE KEM over DiTRU^{ϕ⁺} demonstrate performance that is nearly equivalent to that of NTRU-HPS from the PPKE when it comes to the encapsulation process.

- **Decapsulation:** The decapsulation process in E-DiTRU⁺ offers the fastest performance, similar to encapsulation. Moreover, DiTRU⁺ performs comparably to the decapsulation of the PPKE KEM derived from NTRU-HPS. In contrast, the decapsulation process for the DPKE KEM over DiTRU^{ϕ⁺} is slower than that of the PPKE KEM over NTRU-HPS and performs similarly to the DPKE KEM over NTRU-HPS.

Analysis. The KeyGen, Encap, and Decap processes are primarily dominated by “polynomial multiplication”. Therefore, we will examine the costs associated with these multiplications in relation to the processes and the length of the polynomial needed to achieve equivalent levels of security across the various schemes.

For *KeyGen*, we consider the cost with respect to the constant time implementation as per the Bernstein-Yang algorithm [75]. For NTRU-HPS, as q is selected as a power of 2, the inverse of f is computed in the ring $\mathbb{Z}_2[x]/(x^N - 1)$ and then is lifted to the $\mathbb{Z}_q[x]/(x^N - 1)$ using the almost inverse algorithm described in [71]. Therefore, the total cost of inverting one polynomial is around 8 polynomial multiplications. Further, one extra polynomial multiplication is performed to compute the public key as $f^{-1} * g \pmod{q}$. Hence, the total number of polynomial multiplications for NTRU-HPS is almost 9. For DiTRU and DiTRU^{ϕ⁺}, the key generation process involves transforming the problem of inverting elements over the corresponding ring into finding inverses over a group ring of a cyclic group. This transformation uses a “polynomial” that has half the length of the original polynomial, along with four multiplications, as detailed in Algorithm 1. Consequently, for DiTRU⁺, the total cost of calculating the inverse (in constant time) amounts to 12 multiplications of polynomials with half the length of the original polynomial plus one additional multiplication of the original polynomial length for the public key formula. In DiTRU^{ϕ⁺}, the key generation process involves two inversions modulo q (equivalent to 24 multiplications of length $N/2$) and one inversion modulo p (equivalent to 4 multiplications of length $N/2$).

Therefore, the total cost amounts to 28 multiplications of length $N/2$ plus one multiplication of length N for the public key formula.

For Ei-TRU and Ei-DiTRU⁺, the modulo q is a prime number. In the case of Ei-TRU, the cost of inverting an element is translated into inverting a polynomial over $\mathbb{Z}[w]/\langle q \rangle C_{N/4}$, that is one-third the dimension of the original polynomial plus 15 polynomial multiplications of one-third the size of the input polynomial [33, Algorithm 2], along with one additional multiplication for the key formula. On the other hand, for Ei-DiTRU⁺, the inversion process requires four multiplications of half the dimensions of the input polynomial. Additionally, it includes the cost of inverting a polynomial $\in \mathbb{Z}[w]/\langle q \rangle C_{N/4}$ that is half the dimension of the input polynomial, calculated modulo the prime q . The inversion process for a polynomial $\in \mathbb{Z}[w]/\langle q \rangle C_{N/4}$, in relation to the prime q , requires the equivalent of 19 polynomial multiplications. It is important to note that all these multiplications are conducted within the ring of Eisenstein integers for both Ei-TRU and Ei-DiTRU⁺.

For *Encapsulation*, all considered KEMs require one polynomial multiplication of the input polynomial length. For *Decapsulation*, all KEMs require two polynomial multiplications, except for the DPKE KEM over DiTRU^{ϕ⁺}, which requires three polynomial multiplications. Table 9 compares the cost of the KeyGen, Encapsulation, and Decapsulation for the KEMs considered in this paper with respect to the required polynomial multiplications and the number of scalar multiplications. One can observe that this analysis aligns with the experimental results reported in Table 3 and Figure 13.

6. Conclusion

The flexibility of the NTRU framework allows for the creation of various variants built on different rings. This paper presents NTRU KEMs using noncommutative algebra derived from twisted dihedral group rings. Two of the KEMs are based on PPKE, while the third is constructed from DPKE. We provide performance measurements for both the ring of integers and the ring of Eisenstein integers. Our results show that the proposed schemes perform effectively in both encapsulation and decapsulation when using the ring of integers, and they exhibit a notably fast key generation process. Additionally, performance improves further when employing the ring of Eisenstein integers.

Overall, our findings indicate that our constructions are efficient and compact, leveraging a noncommutative ring as the foundational algebra for the NTRU variants.

References

- [1] P. W. Shor, Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer, *SIAM Journal on Computing* 26 (5) (1997) 1484–1509. doi: 10.1137/S0097539795293172. URL <http://dx.doi.org/10.1137/S0097539795293172>

Table 9: Theoretical comparison of the costs of Key Generation, Encapsulation, and Decapsulation in terms of polynomial multiplications performed per primitive.

	poly	KeyGen		Encap		Decap	
		#mul	cost	#mul	cost	#mul	cost
NTRU-HPS	N	$9(\text{len}N)$	$9N^2$	$1(\text{len}N)$	N^2	$2(\text{len}N)$	$2N^2$
DiTRU	$\approx 2N$	$12(\text{len}N) + 1(\text{len}2N)$	$16N^2$	$1(\text{len}2N)$	$4N^2$	$2(\text{len}2N)$	$8N^2$
Ei-TRU	$\approx \frac{3}{4}N(\text{over Eisenstein})$	$34(\text{len}\frac{N}{4}) + 1(\text{len}\frac{3}{4}N)$	$\approx 8N^2$	$1(\text{len}\frac{3}{4}N)$	$\approx 1.7N^2$	$2(\text{len}\frac{3}{4}N)$	$\approx 3.4N^2$
DiTRU ⁺	$\approx N$	$12(\text{len}\frac{N}{2}) + 1(\text{len}N)$	$4N^2$	$1(\text{len}N)$	N^2	$2(\text{len}N)$	$2N^2$
Ei-DiTRU ⁺	$\approx \frac{1}{2}N(\text{over Eisenstein})$	$23(\text{len}\frac{N}{4}) + 1(\text{len}\frac{N}{2})$	$\approx 5N^2$	$1(\text{len}\frac{N}{2})$	$0.75N^2$	$2(\text{len}\frac{N}{2})$	$1.5N^2$
DiTRU ^{φ+}	$\approx N$	$28(\text{len}\frac{N}{2}) + 1(\text{len}N)$	$\approx 8N^2$	$1(\text{len}N)$	N^2	$3(\text{len}N)$	$3N^2$

- [2] J. Proos, C. Zalka, Shor’s discrete logarithm quantum algorithm for elliptic curves (2004). [arXiv:quant-ph/0301141](https://arxiv.org/abs/quant-ph/0301141). URL <https://arxiv.org/abs/quant-ph/0301141>
- [3] M. Roetteler, M. Naehrig, K. M. Svore, K. Lauter, Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms, in: T. Takagi, T. Peyrin (Eds.), *Advances in Cryptology – ASIACRYPT 2017*, Springer International Publishing, Cham, 2017, pp. 241–270. doi:10.1007/978-3-319-70697-9_9.
- [4] C. NIST, Submission requirements and evaluation criteria for the post-quantum cryptography standardization process (2016). URL <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>
- [5] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, et al., Status report on the third round of the NIST post-quantum cryptography standardization process, Tech. rep., National Institute of Standards and Technology (NIST) (2022). URL <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413.pdf>
- [6] C. NIST, Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process (2022). URL <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>
- [7] J. Hoffstein, J. Pipher, J. H. Silverman, NTRU: A ring-based public key cryptosystem, in: *International algorithmic number theory symposium*, Springer, Berlin, Heidelberg, 1998, pp. 267–288. doi:10.1007/BFb0054868.
- [8] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, Z. Zhan, FALCON: Fast-Fourier lattice-based compact signatures over NTRU, Tech. rep., National Institute of Standards and Technology (NIST), <https://www.di.ens.fr/~prest/Publication/s/falcon.pdf> (2018).
- [9] C. Chen, O. Danba, J. Hoffstein, A. Hülsing, J. Rijneveld, J. M. Schanck, P. Schwabe, W. Whyte, Z. Zhang, NTRU: Algorithm specifications and supporting documentation, NIST (2020). URL <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
- [10] C. Chen, J. Hoffstein, W. Whyte, Z. Zhang, NIST PQ Submission: NTRUEncrypt A lattice based encryption algorithm, NIST (2017). URL <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions>
- [11] A. Hülsing, J. Rijneveld, J. M. Schanck, P. Schwabe, NTRU-HRSS-KEM: Algorithm specifications and supporting documentation, NIST submission 30 (2017). URL <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions>
- [12] D. J. Bernstein, C. Chuengsatiansup, T. Lange, C. van Vredendaal, NTRU Prime: Reducing Attack Surface at Low Cost, in: *Selected Areas in Cryptography – SAC 2017*, Springer International Publishing, 2018, pp. 235–260. doi:10.1007/978-3-319-72565-9_12.
- [13] Working Group of the C/MM Committee and others, IEEE P1363.1 Standard Specification for Public-Key Cryptographic Techniques Based on Hard Problems over Lattices, <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=4800404> (2009).
- [14] C. Peikert, A decade of lattice cryptography, *Found. Trends Theor. Comput. Sci.* 10 (4) (2016) 283–424. URL <https://doi.org/10.1561/04000000074>
- [15] S. Agrawal, Indistinguishability Obfuscation Without Multilinear Maps: New Methods for Bootstrapping and Instantiation, in: *Advances in Cryptology – EUROCRYPT 2019*, Springer International Publishing, Cham, 2019, pp. 191–225. doi:10.1007/978-3-030-17653-2_7.
- [16] S. Agrawal, A. Pellet-Mary, Indistinguishability obfuscation without maps: Attacks and fixes for noisy linear FE, in: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2020, pp. 110–140. doi:10.1007/978-3-030-45721-1_5.
- [17] J. Kim, C. Lee, A polynomial time algorithm for breaking NTRU encryption with multiple keys, *Designs, Codes and Cryptography* 91 (2023) 2779–2789. doi:10.1007/s10623-023-01233-5.
- [18] S. Bai, H. Jangir, T. Ngo, W. Youmans, An algebraic algorithm for breaking NTRU with multiple keys, *Designs, Codes and Cryptography* 92 (2024) 3895–3918. doi:10.1007/s10623-024-01473-z.
- [19] J. Hoffstein, J. H. Silverman, A non-commutative version of the NTRU public key cryptosystem, unpublished paper, February (1997).
- [20] D. Coppersmith, A. Shamir, Lattice Attacks on NTRU, in: *Advances in Cryptology — EUROCRYPT ’97*, Springer Berlin Heidelberg, Berlin, Heidelberg, 1997, pp. 52–61. doi:10.1007/3-540-69053-0_5.
- [21] K. R. Truman, Analysis and extension of non-commutative NTRU, PhD dissertation, University of Maryland (2007). URL <https://drum.lib.umd.edu/handle/1903/7344>
- [22] N. Vats, NNRU, a noncommutative analogue of NTRU (2009). [arXiv:0902.1891](https://arxiv.org/abs/0902.1891). URL <https://doi.org/10.48550/arXiv.0902.1891>
- [23] A. Hassani Karbasi, S. Ebrahimi Atani, R. Ebrahimi Atani, PairTRU: Pairwise Non-commutative Extension of The NTRU Public key Cryptosystem, *International Journal of Information Security Science* 8 (2018) 1–10.
- [24] E. Malekian, A. Zakerolhosseini, A. Mashatan, QTRU : a lattice attack resistant version of NTRU PKCS based on quaternion algebra, *IACR Cryptology ePrint Archive* 386 (2009). URL <https://eprint.iacr.org/2009/386>
- [25] K. Thakur, A variant of NTRU with split quaternions algebra, *Palestine J. of Mathematics* 6 (2) (2017) 598–610. URL https://pjm.ppu.edu/sites/default/files/papers/PJM_April_2017_28.pdf
- [26] E. Malekian, A. Zakerolhosseini, OTRU: A non-associative and high speed public key cryptosystem, in: *2010 15th CSI International Symposium on Computer Architecture and Digital Systems*, 2010, pp. 83–90. doi:10.1109/CADS.2010.5623536.
- [27] K. Bagheri, M.-R. Sadeghi, D. Panario, A non-commutative

- cryptosystem based on quaternion algebras, *Designs, Codes and Cryptography* 86 (2018) 2345–2377. doi:10.1007/s10623-017-0451-4.
- [28] A. Raya, V. Kumar, A. K. Gangopadhyay, S. Gangopadhyay, Giant Does NOT Mean Strong: Cryptanalysis of BQTRU, in: R. Niederhagen, M.-J. O. Saarinen (Eds.), *Post-Quantum Cryptography*, Springer Nature Switzerland, Cham, 2025, pp. 312–348. doi:10.1007/978-3-031-86599-2_11.
 - [29] C. Gentry, Key recovery and message attacks on NTRU-composite, in: B. Pfitzmann (Ed.), *Advances in Cryptology — EUROCRYPT 2001*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 182–194. URL https://doi.org/10.1007/3-540-44987-6_12
 - [30] C. Ling, A. Mendelsohn, NTRU in quaternion algebras of bounded discriminant, in: *Post-Quantum Cryptography*, Springer Nature Switzerland, Cham, 2023, pp. 256–290. doi:10.1007/978-3-031-40003-2_10.
 - [31] A. Raya, V. Kumar, A. K. Gangopadhyay, DiTRU: A Resurrection of NTRU over Dihedral Group, in: S. Vaudenay, C. Petit (Eds.), *Progress in Cryptology - AFRICACRYPT 2024*, Springer Nature Switzerland, Cham, 2024, pp. 349–375. URL [10.1007/978-3-031-64381-1_16](https://doi.org/10.1007/978-3-031-64381-1_16)
 - [32] V. Kumar, R. Das, A. K. Gangopadhyay, GR-NTRU: Dihedral group over ring of Eisenstein integers, *Journal of Information Security and Applications* 83 (2024) 103795. doi:<https://doi.org/10.1016/j.jisa.2024.103795>.
 - [33] V. Kumar, A. Raya, A. K. Gangopadhyay, S. Gangopadhyay, M. T. Hussain, An Efficient Noncommutative NTRU from Semidirect Product, in: S. Mukhopadhyay, P. Stănică (Eds.), *Progress in Cryptology – INDOCRYPT 2024*, Springer Nature Switzerland, Cham, 2025, pp. 3–27. doi:10.1007/978-3-031-80308-6_1.
 - [34] C. Chen, O. Danba, J. Hoffstein, A. Hülsing, J. Rijneveld, J. M. Schanck, T. Saito, P. S. Schwade, W. W. Whyte, K. X. Xagawa, T. Yamakawa, Z. Zhang, PQC round-3 candidate: NTRU. technical report, Tech. rep., NTRU Cryptosystems Technical Report No.11, Version 2, March 2001. Report (2019). URL <https://ntru.org/f/ntru-20190330.pdf>
 - [35] N. Howgrave-Graham, J. H. Silverman, W. Whyte, Choosing parameter sets for NTRUencrypt with NAEP and SVES-3, in: A. Menezes (Ed.), *Topics in Cryptology – CT-RSA 2005*, 2005, pp. 118–135. doi:10.1007/978-3-540-30574-3_10.
 - [36] N. Fox, Spectra of Semidirect Products of Cyclic Groups, *Rose-Hulman Undergraduate Mathematics Journal* 11 (2010). URL <https://scholar.rose-hulman.edu/rhumj/vol11/iss2/7>
 - [37] J. Don, S. Fehr, C. Majenz, C. Schaffner, Online-extractability in the quantum random-oracle model, in: O. Dunkelman, S. Dziembowski (Eds.), *Advances in Cryptology – EUROCRYPT 2022*, Springer International Publishing, Cham, 2022, pp. 677–706. doi:10.1007/978-3-031-07082-2_24.
 - [38] E. Fujisaki, T. Okamoto, Secure integration of asymmetric and symmetric encryption schemes, in: M. Wiener (Ed.), *Advances in Cryptology — CRYPTO’ 99*, Springer Berlin Heidelberg, Berlin, Heidelberg, 1999, pp. 537–554. doi:10.1007/3-540-48405-1_34.
 - [39] E. Fujisaki, T. Okamoto, Secure integration of asymmetric and symmetric encryption schemes, *Journal of Cryptology* 26 (2013) 80–101. URL <https://doi.org/10.1007/s00145-011-9114-1>
 - [40] D. Hofheinz, K. Hövelmanns, E. Kiltz, A Modular Analysis of the Fujisaki-Okamoto Transformation, in: Y. Kalai, L. Reyzin (Eds.), *Theory of Cryptography*, Springer International Publishing, Cham, 2017, pp. 341–371. doi:10.1007/978-3-319-70500-2_12.
 - [41] S. Bai, H. Jangir, H. Lin, T. Ngo, W. Wen, J. Zheng, Compact Encryption Based on Module-NTRU Problems, in: M.-J. Saarinen, D. Smith-Tone (Eds.), *Post-Quantum Cryptography*, Springer Nature Switzerland, Cham, 2024, pp. 371–405. doi:10.1007/978-3-031-62743-9_13.
 - [42] J. Hoffstein, J. Pipher, J. Silverman, *An Introduction to Mathematical Cryptography*, 1st Edition, Springer Publishing Company, Incorporated, NY, 2008.
 - [43] D. S. Dummit, R. M. Foote, *Abstract Algebra*, 3rd Edition, John Wiley & Sons, Inc, 2003.
 - [44] P. Schwabe, R. Avanzi, J. B. Bos, L. D.ucas, E. Kiltz, T. crede Lepoint, V. Lyubashevsky, J. M. Schanck, G. Seiler, D. Stehle, *Crystals-Kyber*, Tech. rep., National Institute of Standards and Technology (NIST) (2017). URL <https://csrc.nist.gov/projects/post-quantum-cryptography/round-1-submissions>
 - [45] T. Yasuda, X. Dahan, K. Sakurai, Characterizing NTRU-variants using group ring and evaluating their lattice security, *IACR Cryptol. ePrint Arch.* (2015) 1170. URL <http://eprint.iacr.org/2015/1170>
 - [46] T. Hurley, Group rings and rings of matrices, *International Journal of Pure and Applied Mathematics* 31 (2006) 319–335. URL https://www.researchgate.net/publication/228928727_Group_rings_and_rings_of_matrices
 - [47] V. Kumar, A. Raya, S. Gangopadhyay, A. K. Gangopadhyay, Cryptanalysis of Group Ring NTRU: The Case of the Dihedral Group, *Security and Privacy* 8 (2025) 1–15. URL <https://doi.org/10.1002/spy2.70020>
 - [48] J. Hoffstein, J. H. Silverman, W. Whyte, Meet-in-the-middle attack on an NTRU private key, Tech. rep., Technical report, NTRU Cryptosystems, July 2006. Report (2006). URL <https://ntru.org/f/tr/tr004v2.pdf>
 - [49] A. K. Lenstra, H. W. Lenstra, L. Lovász, Factoring polynomials with rational coefficients, *Mathematische annalen* 261 (ARTICLE) (1982) 515–534. doi:10.1007/BF01457454.
 - [50] C. P. Schnorr, A hierarchy of polynomial time lattice basis reduction algorithms, *Theoretical computer science* 53 (2-3) (1987) 201–224. doi:10.1016/0304-3975(87)90064-8.
 - [51] Y. Aono, Y. Wang, T. Hayashi, T. Takagi, Improved progressive BKZ algorithms and their precise cost estimation by sharp simulator, in: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2016, pp. 789–819. doi:10.1007/978-3-662-49890-3_30.
 - [52] E. Alkim, L. Ducas, T. Pöppelmann, P. Schwabe, Post-quantum key {Exchange—A} new hope, in: *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 327–343. URL https://www.usenix.org/system/files/conference/useenixsecurity16/sec16_paper_alkim.pdf
 - [53] U. Fincke, M. Pohst, Improved methods for calculating vectors of short length in a lattice, including a complexity analysis, *Mathematics of computation* 44 (170) (1985) 463–471.
 - [54] A. Becker, L. Ducas, N. Gama, T. Laarhoven, New directions in nearest neighbor searching with applications to lattice sieving, in: *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, SIAM, 2016, pp. 10–24. doi:10.1137/1.9781611974331.ch2.
 - [55] E. Kirshanova, A. May, J. Nowakowski, New NTRU Records with Improved Lattice Bases, in: *Post-Quantum Cryptography*, 2023, pp. 167–195. doi:10.1007/978-3-031-40003-2_7.
 - [56] T. Laarhoven, Search problems in cryptography: from fingerprinting to lattice sieving, Phd thesis, Eindhoven University of Technology (2015). URL <https://research.tue.nl/en/publications/search-problems-in-cryptography-from-fingerprinting-to-lattice-sieving>
 - [57] E. W. Postlethwaite, F. Virdia, On the success probability of solving unique SVP via BKZ, in: J. A. Garay (Ed.), *Public-Key Cryptography – PKC 2021*, Springer International Publishing, Cham, 2021, pp. 68–98. doi:10.1007/978-3-030-75245-3_4.
 - [58] D. Dachman-Soled, L. Ducas, H. Gong, M. Rossi, LWE with Side Information: Attacks and Concrete Security Estimation, in: *Advances in Cryptology – CRYPTO 2020*, Springer International Publishing, Cham, 2020, pp. 329–358. doi:10.1007/978-3-030-56880-1_12.
 - [59] C. Schnorr, A hierarchy of polynomial time lattice basis reduction algorithms, *Theoretical Computer Science* 53 (2) (1987) 201–224. doi:[https://doi.org/10.1016/0304-3975\(87\)90064-8](https://doi.org/10.1016/0304-3975(87)90064-8).

- [60] Y. Chen, Réduction de réseau et sécurité concrète du chiffrement complètement homomorphe, Ph.D. thesis, l'Université Paris Diderot (2013).
URL <http://www.theses.fr/2013PA077242>
- [61] L. Ducas, W. van Woerden, NTRU Fatigue: How Stretched is Overstretched?, in: *Advances in Cryptology – ASIACRYPT 2021*, Springer International Publishing, 2021, pp. 3–32. doi:10.1007/978-3-030-92068-5_1.
- [62] N. Howgrave-Graham, A Hybrid Lattice-Reduction and Meet-in-the-Middle Attack Against NTRU, in: *Advances in Cryptology – CRYPTO 2007*, Springer Berlin Heidelberg, 2007, pp. 150–169. doi:10.1007/978-3-540-74143-5_9.
- [63] J. Hoffstein, J. Pipher, J. M. Schanck, J. H. Silverman, W. Whyte, Z. Zhang, Choosing Parameters for NTRUEncrypt, in: H. Handschuh (Ed.), *Topics in Cryptology – CT-RSA 2017*, 2017. doi:10.1007/978-3-319-52153-4_1.
- [64] M. Albrecht, S. Bai, L. Ducas, A subfield lattice attack on overstretched NTRU assumptions, in: *Advances in Cryptology – CRYPTO 2016*, Springer Berlin Heidelberg, 2016, pp. 153–178. doi:10.1007/978-3-662-53018-4_6.
- [65] J. Cheon, J. Jeong, C. Lee, An algorithm for NTRU problems and cryptanalysis of the GGH multilinear map without a low-level encoding of zero, *LMS Journal of Computation and Mathematics* 19 (2016) 255–266. doi:10.1112/S1461157016000371.
- [66] P. Kirchner, P. A. Fouque, Revisiting lattice attacks on overstretched NTRU parameters, in: *Advances in Cryptology – EUROCRYPT 2017*, Springer International Publishing, 2017, pp. 3–26. doi:10.1007/978-3-319-56620-7_1.
- [67] N. Howgrave-Graham, J. H. Silverman, A. Singer, W. Whyte, NAEP: Provable security in the presence of decryption failures, *Cryptology ePrint Archive* (2003).
URL <https://eprint.iacr.org/2003/172>
- [68] E. Fujisaki, T. Okamoto, Secure integration of asymmetric and symmetric encryption schemes, *Journal of Cryptology* 26 (2013) 80–101.
URL <https://doi.org/10.1007/s00145-011-9114-1>
- [69] A. Hülsing, J. Rijneveld, J. Schanck, P. Schwabe, High-speed key encapsulation from NTRU, in: *International Conference on Cryptographic Hardware and Embedded Systems, CHES 2017*, 2017, pp. 232–252. doi:10.1007/978-3-319-66787-4_12.
- [70] H. Wu, L. Zhu, R. Feng, S. Yang, Explicit factorizations of cyclotomic polynomials over finite fields, *Designs, Codes and Cryptography* 83 (2017) 197–217.
URL <https://doi.org/10.1007/s10623-016-0224-5>
- [71] J. H. Silverman, Almost inverses and fast NTRU key creation, NTRU Cryptosystems Technical Report #14 (1999).
URL <https://ntru.org/f/tr/tr014v1.pdf>
- [72] T. Saito, K. Xagawa, T. Yamakawa, Tightly-secure key-encapsulation mechanism in the quantum random oracle model, in: J. B. Nielsen, V. Rijmen (Eds.), *Advances in Cryptology – EUROCRYPT 2018*, Springer International Publishing, Cham, 2018, pp. 520–551. doi:10.1007/978-3-319-78372-7_17.
- [73] D. J. Bernstein, E. Persichetti, Towards KEM unification, *Cryptology ePrint Archive* (2018).
URL <https://eprint.iacr.org/2018/526>
- [74] K. Jarvis, M. Nevins, ETRU: NTRU over the eisenstein integers, *Des. Codes Cryptogr.* 74 (2015) 219–242.
URL <https://doi.org/10.1007/s10623-013-9850-3>
- [75] D. J. Bernstein, B.-Y. Yang, Fast constant-time gcd computation and modular inversion, *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019 (3) (2019) 340–398. doi:10.13154/tches.v2019.i3.340-398.